

Survey Paper

A survey of Boolean operations in 3D geometric modeling

Sili Liang^a, Anchang Bao^b, Enya Shen^{b,*}, Jianmin Wang^b^a College of Software, Nankai University, Tianjin, China^b School of Software, Tsinghua University, Beijing, China

ARTICLE INFO

Keywords:

Boolean operations
Geometric modeling
Computer aided design

ABSTRACT

In this article, we provide a detailed survey of existing Boolean operations in 3D modeling. We summarize the core computational procedures of most Boolean operation algorithms into a four-stage conceptual pipeline, comprising intersection computation, geometric splitting, fragment classification, and topological reconstruction. We also discuss alternative approaches that fall outside this pipeline, including CSG, BSP, volumetric, implicit, and hybrid representations. Furthermore, we examine three critical and long-standing challenges in Boolean operations: robustness and numerical stability, efficiency and scalability, and extensibility and generalization. This survey synthesizes foundational techniques, recent advancements, and emerging trends to serve as a comprehensive reference for researchers and practitioners developing robust, efficient, and extensible Boolean operations in modern 3D geometric modeling systems. To the best of our knowledge, this is the first dedicated, standalone survey on this topic.

1. Introduction

Boolean operations are classical and foundational in 3D geometric modeling. Rooted in Boolean algebra, they apply set-theoretic principles to solids, surfaces, and curves, providing one of the most direct and intuitive ways to compose and modify volumetric regions. This explicit control over solid occupancy underpins a wide range of tasks in computer-aided design (CAD) [1], computer graphics [2], and digital manufacturing [3,4]. Other modeling strategies, such as feature-based operations, deformation or direct-editing tools, and procedural or history-based approaches, are powerful and widely used, but they cannot fully replace Boolean operations. Many of these higher-level modeling tools ultimately invoke Boolean operations directly, or rely on Boolean evaluation indirectly when modifying, trimming, or composing solid volumes.

Despite their long history, Boolean operations remain an active technical topic in geometric modeling, attracting continued academic interest [5–12] and ongoing industrial development in geometric kernels such as Parasolid [13], ACIS [14], OpenCASCADE [15], and CGAL [16, 17]. Fig. 1 shows the publication trend from 1984 to 2025 across major venues. Although the number of studies fluctuates over time, the overall trend indicates that Boolean operations remain an active and evolving research direction.

Meanwhile, achieving efficient and robust implementations of Boolean operations continues to pose challenges due to numerical in-

stabilities from floating-point arithmetic, topological singularities, and the demands of large-scale geometric complexity. The most persistent and difficult issue lies in robustness, ensuring consistent and valid results under finite precision arithmetic. Even minor numerical errors can lead to topological degeneracies, such as gaps, self-intersections, or non-manifold boundaries, which are unacceptable in downstream applications like simulation or manufacturing. Moreover, efficiency and scalability become critical when handling complex geometric models, performing batch operations, or supporting real-time editing environments. While many modern algorithms address local improvements, few can guarantee both robustness and performance at scale.

These issues reveal that Boolean operations, though conceptually simple, are still far from being a solved problem in geometry modeling. A systematic understanding of how Boolean algorithms are designed, implemented, and optimized across different representations and computational paradigms is therefore crucial for advancing both theoretical and practical developments. Although extensively studied, Boolean operations in 3D geometric modeling still lack a dedicated comprehensive survey. Existing books typically discuss Boolean operations as subsections within broader topics such as solid modeling or geometric processing [18,19]. We fill that gap by offering the first dedicated survey of existing methods, key challenges, and potential research opportunities.

* Corresponding author.

E-mail addresses: sl.liang@mail.nankai.edu.cn (S. Liang), bac24@mails.tsinghua.edu.cn (A. Bao), shenena@tsinghua.edu.cn (E. Shen), jimwang@tsinghua.edu.cn (J. Wang).

<https://doi.org/10.1016/j.cad.2026.104081>

Received 17 November 2025; Received in revised form 11 February 2026; Accepted 29 March 2026

Available online 30 April 2026

0010-4485/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

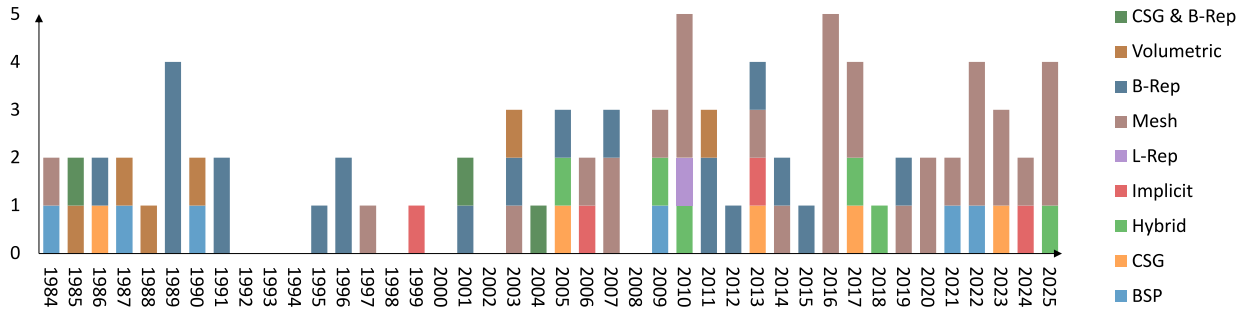


Fig. 1. Publication trends of Boolean operations in 3D geometric modeling across major venues (1984–2025).

Table 1

Scope of the survey: representative papers grouped by representation type.

Representation type	Representative papers
Explicit	
B-Rep	[16,17,28–50]
Mesh	[4,6–8,10,11,51–71]
BSP	[23,72–76]
Volumetric	[77–82]
Implicit	[9,83–86]
CSG	[12,20,28,43,46,87–90]
Hybrid	[5,27,91–94]

1.1. Scope and objectives

This survey focuses on Boolean operations in 3D geometric modeling. It surveys a wide range of algorithmic techniques developed for different model representations from 1984 to 2025, including constructive solid geometry (CSG) [20], boundary representation (B-Rep) [21], polygonal meshes [22], binary space partitioning (BSP) representation [23], implicit representation [24,25], volumetric representation [26], and hybrid representation [5,27]. The emphasis is on core algorithmic and computational aspects, rather than on application-specific adaptations or low-level implementation details. Table 1 outlines the scope of this survey by grouping representative papers according to their representation type and modeling paradigm.

For B-Rep and mesh based approaches, which form the basis of most classical and industrial Boolean algorithms, we organize the literature through a common computational pipeline consisting of intersection evaluation, region classification, and topological reconstruction. For BSP, implicit, voxel, hybrid methods, where a unified pipeline is less well-defined, we provide a comparative summary based on their representational assumptions and algorithmic characteristics.

The survey aims to provide a comprehensive reference for researchers and practitioners developing robust, efficient, and extensible Boolean operations in modern geometric modeling systems.

1.2. Organization

The remainder of this survey is structured as follows. Section 2 provides a brief review of several major modeling representations and Boolean operations from set-theoretic and algebraic-topological perspectives, establishing the essential theoretical background. Section 3 outlines a common four-stage Boolean operation workflow, intersection computation, geometric splitting, fragment classification, and topological reconstruction, highlighting common strategies that are independent of specific geometric representations. Sections 4 surveys Boolean operations for other representations, including CSG, BSP, volumetric representation, implicit representation, and hybrid representation, highlighting their unique algorithmic characteristics and practical applications. Section 5 synthesizes key challenges in robustness, efficiency, and generalization by assessing existing solutions,

pinpointing open problems, and proposing future research directions. Finally, Section 6 summarizes the main insights, reflects on the current state of the field, and offers guidance for future research.

2. Background

2.1. Major model representations

In solid modeling, a variety of geometric representations have been proposed to define shapes and implement modeling algorithms. A solid representation encodes both the geometry and topology of physical objects in a computer-interpretable form, and different schemes exhibit trade-offs in expressiveness, precision, and computational cost.

Historically, early foundational work played a significant role in shaping these approaches. The PROLAMAT '73 proceedings brought together several pioneering contributions that introduced and compared major classes of solid modeling methods that later influenced CAD system design. Among these, B-Rep were explored by Braid and colleagues through the BUILD system presented at the same conference. Similarly, Okino and collaborators presented the TIPS system [95], one of the earliest operational frameworks based on half-space constructive modeling.

This subsection introduces six major classes of representation used in solid modeling: CSG, B-Rep, polygonal mesh, BSP representation, volumetric representation, and implicit representation. Fig. 2 illustrates five of these representation types.

Constructive solid geometry (CSG). CSG represents solids by recursively combining primitive shapes using Boolean operations [20]. These primitives, such as spheres, boxes, and cylinders, are organized into a binary operation tree, with leaf nodes representing primitives and internal nodes representing Boolean operations like union, intersection, and difference. The strengths of CSG include its simplicity, compactness, and suitability for parametric modeling. However, it lacks direct control over surface details and topological structure, making it unsuitable for complex free-form geometry or precise boundary manipulations.

Boundary representation (B-Rep). B-Rep originated in the early 1970s in the work of Braid and the BUILD group [21,96]. Braid's early writings laid out the core ideas of representing solids by faces, edges, and vertices, together with requirements for topological consistency and Boolean evaluation. Later work by Requicha, Voelcker, and others helped formalize and broaden these ideas [28].

B-Rep explicitly describes a solid object in terms of its boundary surfaces [21]. It encodes geometry through vertices, edges, and faces, and topology through the adjacency relationships among these elements. Owing to its geometric accuracy and support for local operations, B-Rep became the dominant representation in CAD/CAM and underlies a sequence of influential modeling kernels and systems, including ROMULUS, ACIS [14], Parasolid [13], Dassault CGM [97], and PTC Granite [98]. However, Boolean operations on B-Rep require careful handling of surface–surface intersections, region classification, and reconstruction of valid topology, making them computationally demanding and numerically sensitive [28].

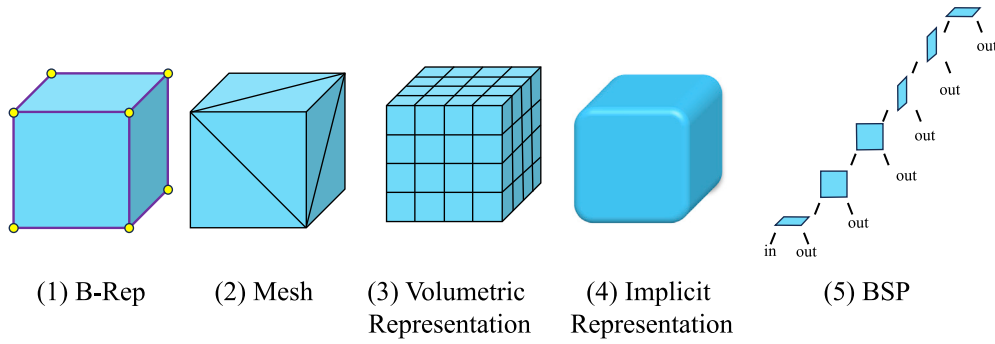


Fig. 2. Major model representations: (1) B-Rep, (2) Mesh, (3) Volumetric, (4) Implicit, (5) BSP.

Polygonal meshes. Polygonal meshes may be regarded as a special kind of boundary representation, as they describe solids through surface geometry and adjacency. However, we discuss meshes separately from B-Rep models because their geometric assumptions, numerical behavior, and pipelines differ substantially in practice. Meshes discretize object boundaries into collections of polygons [22], typically triangles or quadrilaterals. This representation is common in computer graphics, 3D scanning, and visualization systems. Meshes provide a lightweight approximation of geometry and are well-suited for GPU processing and real-time applications. Boolean operations on triangle meshes are commonly implemented by computing triangle-triangle intersections, splitting faces along the resulting curves, classifying the fragments, and reconstructing the output topology. With robust predicates, adaptive precision, or arrangement-based reconstruction, mesh Booleans can preserve watertightness and topological validity. However, tolerance-based or purely floating-point implementations may suffer from missed intersections, small gaps, or non-manifold artifacts, particularly on noisy, low-resolution, or inconsistent inputs.

Binary Space Partitioning (BSP). Binary Space Partitioning (BSP) represents solid geometry by recursively partitioning 3D space using planes [23,72,73]. Each internal node of the BSP tree defines a splitting plane, dividing space into positive and negative subregions, thereby providing an implicit encoding of the object's topology. This hierarchical organization allows Boolean operations to be performed through tree traversal and merging, enabling constructive modeling without directly computing explicit surface intersections. The combinatorial and symbolic nature of BSP makes it particularly suitable for exact and topologically consistent results, even when dealing with intersecting or non-manifold geometries. However, the efficiency and scalability of BSP representations remain challenging. Complex or high-resolution models can produce unbalanced or excessively large trees, resulting in significant memory overhead and traversal inefficiencies.

Volumetric representation. Volumetric representations describe solids by discretizing the embedding space into a regular or hierarchical grid [26], such as voxel arrays or octrees. Each volumetric cell (voxel) stores scalar attributes—commonly occupancy indicators or signed distance values—that implicitly define the object's interior and boundary. A point is considered inside the solid if its value satisfies a predefined criterion, such as being below zero in the case of a signed distance field. This paradigm enables robust and resolution-independent Boolean operations, as set-theoretic combinations (e.g., union, intersection, difference) can be performed directly on the voxel-wise or field-wise values using simple arithmetic or logical rules.

Implicit representation. Implicit representations define solid geometry through isosurface or zero-set of continuous scalar functions [24,25], such as signed distance functions (SDFs) or neural implicit fields [99]. A point in space is considered to be inside the solid if its function value is below a threshold (typically zero). Implicit surfaces offer strong advantages for shape blending, Boolean operations, and smooth transitions,

as these reduce to simple function combinations (e.g., min/max or soft variants). These methods are especially powerful for representing complex free-form shapes and volumetric data. Notably, implicit surfaces themselves provide an exact continuous representation of geometry; the limitation lies not in surface accuracy but in the absence of explicit edge and face structures, and may require costly extraction methods, such as Marching Cubes [100,101], to recover meshes for visualization or downstream processing.

2.2. Boolean operations foundation

To lay a foundation for understanding Boolean operations, this subsection introduces several fundamental concepts. A *solid object* in geometric modeling is typically a closed, bounded, and regular subset of three-dimensional Euclidean space $\mathbb{R}^3$. These solids are assumed to be regularized, meaning that they contain all their interior points and exclude non-manifold or lower-dimensional features. Topologically, the boundary of a well-defined solid is often required to be a 2-manifold, each point on the surface has a neighborhood homeomorphic to a disk, to ensure validity in modeling and analysis.

In addition to these classical geometric assumptions, this survey also includes an algebraic-topological paradigm for Boolean operations. In this viewpoint, Boolean evaluation is reinterpreted as a linear-algebraic computation over space arrangements and chain complexes. This paradigm is structurally different from traditional geometry kernels such as B-Rep or mesh-based systems: it evaluates Boolean formulas by operating directly on the atoms of the space arrangement and their binary incidence structure, exploiting the isomorphism between chain complexes and finite Boolean algebras. This perspective becomes advantageous when robustness, combinatorial consistency, or uniform treatment of non-manifold configurations is required, and its computational model is closer to logic circuits than to classical surface-based evaluation. We therefore treat this as a distinct computational framework and introduce it formally in Section 2.2.2.

2.2.1. Set theoretic view

The set-theoretic approach provides a mathematical formalism for Boolean operations by viewing solids as sets of points in $\mathbb{R}^3$. The Boolean operations, like union, intersection, and difference, can be interpreted as set operations:

- **Union** ($A \cup B$): All points belonging to either solid A or solid B .
- **Intersection** ($A \cap B$): Points common to both A and B .
- **Difference** ($A \setminus B$): Points in A but not in B .

Regularized Boolean operations. In geometric modeling, Boolean operations must yield results that are valid solids, as Fig. 3. This leads to the concept of *regularized* Boolean operations, which are defined as follows [102]:

$$A \cup^* B = \text{reg}(A \cup B), \quad A \cap^* B = \text{reg}(A \cap B), \quad A \setminus^* B = \text{reg}(A \setminus B)$$

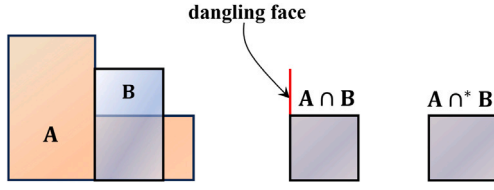


Fig. 3. The regularized intersection of two r -sets is an r -set, but their conventional intersection need not be regular [102,103].

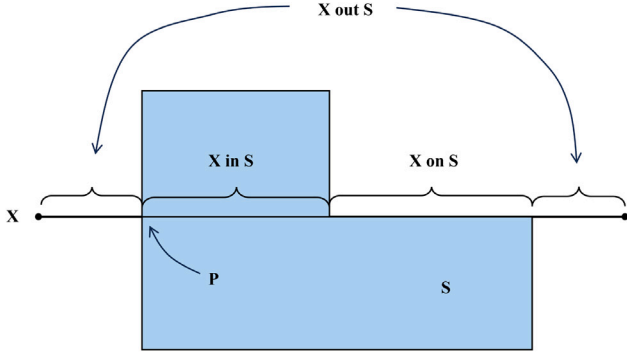


Fig. 4. Set membership classification: Decomposing each solid into **in** and **out** regions relative to the other [29,105].

Here, $\text{reg}(\cdot)$ denotes the regularization of a set, which involves taking the closure of its interior:

$$\text{reg}(S) = \overline{\text{int}(S)}$$

Regularized operations ensure the resulting solid is closed and free of lower-dimensional artifacts, thus ensuring a well-defined result suitable for modeling and fabrication.

Set Membership Classification. Set Membership Classification (SMC) is a technique used to determine the spatial relationship of a point with respect to a solid object [104], as Fig. 4. For a given point $p \in \mathbb{R}^3$, SMC answers whether p lies:

- **Inside** the solid: $p \in \text{int}(S)$
- **On the boundary** of the solid: $p \in \partial S$
- **Outside** the solid: $p \notin \bar{S}$

Efficient and robust SMC is critical to ensure accurate and reliable Boolean computation.

On/on ambiguities and neighborhood resolution. While SMC provides a foundational mechanism to evaluate candidate geometric elements (points, edges, faces) with respect to solids, it encounters an issue in scenarios where a candidate lies on the boundary of both solids. These cases give rise to what are known as on/on ambiguities [28].

Consider a candidate set X that is simultaneously classified as **on** with respect to two solids A and B . That is,

$$X \in \partial A \cap \partial B.$$

In such cases, it is insufficient to determine the classification of X with respect to a Boolean result $S = A \circ B$ (where $\circ$ denotes a regularized Boolean operator such as union, intersection, or difference). As illustrated in Fig. 5, two configurations can yield identical classifications (on/on), but produce different results. These configurations are geometrically distinct: in the upper case, the outward normals of A and B at X are opposite, whereas in the lower case they are in the same direction, as noted by Braid and of Cambridge [96]. Consequently, in one case X lies inside S , and in the other it remains on the boundary.

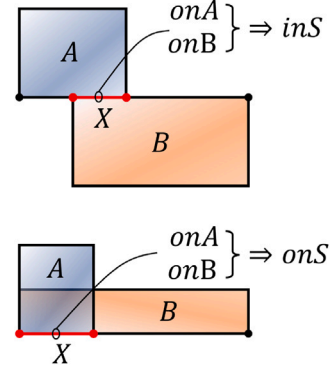


Fig. 5. Two distinct on/on configurations. In the upper case, the outward normals of the two solids at X are opposite; in the lower case, they are in the same direction.

Source: Adapted from [28].

To robustly resolve these ambiguities, Requicha and Voelcker [28] introduced the concept of *neighborhood-based classification*. Instead of only recording the positional relation of X with respect to each operand, the augmented classification also includes the local topological structure around X . Formally, for each point $p \in X$, a neighborhood $N(p, S; R)$ is defined as the regularized intersection between a ball $B(p; R)$ and the solid:

$$N(p, S; R) = B(p; R) \cap^* S.$$

Using the regularized Boolean formulation, this can be decomposed as:

$$N(p, S; R) = N(p, A; R) \circ N(p, B; R).$$

Hence, the classification of X with respect to S can be inferred by combining the neighborhoods of X under A and B via the Boolean operator $\circ$.

The augmented classification of a candidate X with respect to S is thus defined as:

$$X_{\text{wrt } S} = (X_{\text{in}}, (X_{\text{on}}, N(X_{\text{on}}, S)), X_{\text{out}}).$$

By incorporating local neighborhoods, the on/on ambiguity is resolved, allowing for accurate and consistent classification of boundary elements in Boolean evaluation.

This neighborhood-based resolution is not only theoretically elegant but also underpins the correctness of many robust Boolean operation algorithms. It ensures that the semantics of Boolean operations over solids remain well-defined even when input boundaries coincide or overlap. In practice, this step also provides the regularization needed to eliminate lower-dimensional or inconsistently classified regions, completing the classification stage and producing a valid solid suitable for reconstruction.

2.2.2. Algebraic topology view

From a set-theoretic perspective, Boolean operations are viewed as manipulations of regularized point sets, emphasizing the spatial relationships among the involved sets. A natural intermediate viewpoint is the arrangement induced by the input solids: the arrangement decomposes space into elementary cells (“atoms”), and any Boolean solid expression can be described as a selection of these atoms. In this sense, the input shapes act as algebraic generators, and the output corresponds to evaluating a Boolean function over a binary encoding of atom membership, yielding a canonical decomposition of the result [12, 50].

This arrangement-based perspective motivates the algebraic-topological formulation, in which solids are represented as algebraic objects and Boolean operations are reformulated as transformations on chain

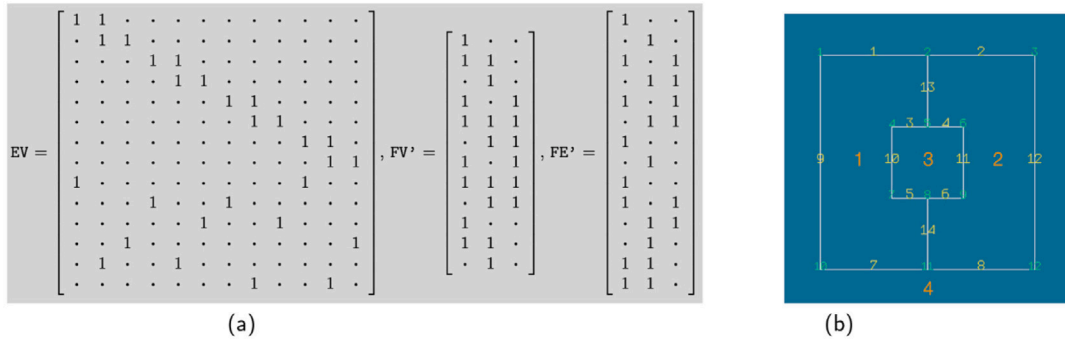


Fig. 6. LAR example [12]. (a) Sparse matrices of selected topological operators. (b) The resulting 2D cellular complex.

complexes [12,34,50]. The following paragraphs introduce the theoretical foundations of this paradigm. The core concepts include chain complexes, boundary operators, and sparse linear algebraic representations that encode how cells in different dimensions are incident to one another.

Cellular decompositions and chain complexes. In algebraic topology, a cell complex is a finite collection of p -dimensional geometric elements, called p -cells, whose interiors are pairwise disjoint and whose closures intersect, if at all, only along common faces. Typical examples include vertices (0-cells), edges (1-cells), faces (2-cells), and volumes (3-cells). Within this framework, a solid is represented as a 3-chain $c \in C_3$, i.e., a linear combination of 3-cells in a graded vector space.

The boundary of a k -chain is obtained via a linear boundary operator $\partial_k : C_k \rightarrow C_{k-1}$, and the identity $\partial_{k-1} \circ \partial_k = 0$ ensures that boundaries of boundaries vanish. Dually, the coboundary operator $\delta^k : C^k \rightarrow C^{k+1}$ satisfies $\delta^{k+1} \circ \delta^k = 0$, and in the discrete (matrix) setting it is represented by the transpose of the boundary operator, i.e., $\delta^k = \partial_{k+1}^T$. These operators assemble into the chain and cochain complexes

$$C_3 \xrightarrow{\partial_3} C_2 \xrightarrow{\partial_2} C_1 \xrightarrow{\partial_1} C_0.$$

making explicit how cells in different dimensions are incident to one another.

Boolean algebras from space arrangements. Given a finite set of solid primitives, they partition the space into sign-invariant regions, known as atoms, where each point has a fixed in/out classification with respect to this set. These atoms define the basis of a finite Boolean algebra $\mathcal{B}$, closed under union, intersection, and complement. To make the symbolic formulation practical and efficient, chain spaces and boundary operators are encoded using sparse matrices. In particular, the Linear Algebraic Representation (LAR) framework [12,50] provides a matrix-based implementation of chain complexes and their algebraic operations (see Fig. 6 for a schematic illustration).

In LAR, a local arrangement is first constructed in 2D for each face after splitting, and then embedded in 3D to assemble the general operator δ_1 . This operator is subsequently used to compute the unknown δ_2 through the Topological Gift Wrapping (TGW) algorithm, whose matrix stores, by columns, the atomic cells induced by the space arrangement of the input objects.

Fig. 6 illustrates a simple 2D example of this representation. The sparse matrices (shown in dense form) corresponding to the topological operators δ_0 , K_2^T , and δ_1 are reported in Fig. 6(a), while the resulting cellular complex is shown in Fig. 6(b). The generated complex consists of $\gamma_0 = 12$ vertices, $\gamma_1 = 14$ edges, and $\gamma_2 = 4$ faces, including the exterior face, yielding an Euler characteristic $\gamma = 2$.

3. General pipeline for explicit Boolean operations

Boolean operations in 3D modeling are usually carried out through a sequence of computational stages. This process-oriented view is not

new. Variants can be found in CAD kernel documentation and technical reports [13–15]. Common descriptions typically involve four essential stages: intersection computation, geometric splitting, fragment classification, and topological reconstruction.

Here, we adopt these four stages as the organizing principle of this section. The perspective focusing on the process rather than on representation types clarifies where robustness issues arise, where efficiency bottlenecks appear, and how different strategies address them.

3.1. Common workflow

Boolean operations over explicit boundary-based representations, such as B-Rep and triangle meshes, follow a broadly shared computational workflow (see Fig. 7). Despite variations in data structures, numerical precision, and application goals, these explicit methods typically rely on a four-stage pipeline that transforms two input solids into a valid output solid by systematically handling intersection computation, splitting geometry, classifying fragments, and reconstructing topology. This pipeline characterizes the core algorithmic challenges of Boolean evaluation in boundary-based systems. It is important to note that this pipeline arises from the explicit face-edge topology required by boundary-based representations. Other modeling paradigms such as implicit fields or voxel grids do not rely on explicit edges or surface subdivisions and therefore avoid many of the topological bookkeeping challenges that motivate this pipeline. Fig. 7 illustrates the general four-stage pipeline through a CAD model example.

1. **Intersection Computation:** This stage locates all points, curves, and surfaces where the input geometries intersect. For triangle meshes, this typically involves triangle-triangle intersection tests; for B-Rep models, face-face and edge-edge intersections are computed.
2. **Geometric Splitting:** Once intersections are identified, the intersected elements, such as edges or faces, are subdivided along intersection curves or points. This process creates fragments that can be individually classified.
3. **Fragment Classification:** After splitting elements into fragments, each fragment is classified according to its relative position relationship with other input shapes. The goal is to determine whether a fragment lies inside, outside, or on the boundary. Then according to the type of Boolean operation, the corresponding category of fragments is discarded.
4. **Topological Reconstruction:** Finally, the retained fragments must be assembled into a valid shape. This involves merging coincident vertices, stitching edges, and restoring surface continuity.

Although this process is commonly presented as sequential, the stages may overlap or be tightly coupled in practice. For example, intersection computation and geometric splitting may be performed simultaneously, or reconstruction may begin before all fragments are

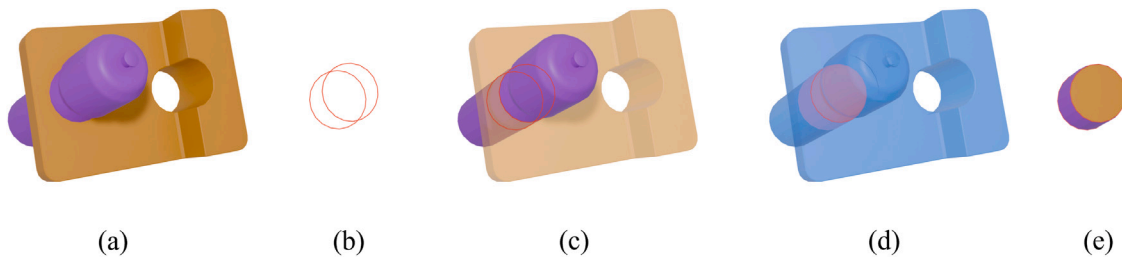


Fig. 7. General computational pipeline for explicit Boolean operations. From left to right: (a) Original models; (b) Intersection computation; (c) Geometry splitting; (d) Fragment classification; (e) Topology reconstruction.

fully labeled. Nevertheless, the four-stage abstraction provides a useful conceptual pipeline for comparing algorithms and understanding the common sources of failure or inefficiency in explicit boundary-based Boolean methods.

3.2. Intersection computation

Intersection computation is the first and essential step in Boolean pipeline. It shares significant similarities with collision detection problems. Both aim to identify and process intersecting geometric elements among a set of candidates. Therefore, the two-stage framework widely used in collision detection [106,107], *broad-phase* for candidate pruning and *narrow-phase* for exact intersection testing, can be naturally adopted here to organize intersection computation techniques.

This subsection focuses on the latter, the computation of intersection elements. The *broad-phase*, while critical for performance, will be discussed in detail in Section 5.2.

Narrow-phase intersection computation is fundamentally determined by the underlying geometric representation. We distinguish two primary categories: mesh-based and parametric-based methods. While mesh-based methods mainly involve triangle-triangle intersection, parametric-based methods necessitate a two-step process comprising geometric intersection followed by topological structuring of intersection curves.

Mesh-based intersection. Triangle-triangle intersection forms the foundational procedure in mesh-based intersection computations. Given two triangles embedded in three-dimensional Euclidean space, the objective is to compute their intersection set, which may be empty, a single point, a line segment, or a polygonal region.

Early research in this area dates back to polygon-clipping algorithms such as the Sutherland-Hodgman algorithm [108] and the Weiler-Atherton method [109], which operated primarily in projected two-dimensional spaces. Although robust and conceptually straightforward, these methods were computationally intensive and numerically sensitive.

The Möller-Trumbore algorithm [110] introduced a simpler predicate-based formulation which was widely adopted for collision detection. By evaluating signed distances of vertices relative to the supporting planes, it detects intersections without explicitly computing intersection geometry. Although originally devised for intersection tests, adaptations of this method have subsequently allowed explicit intersection segment computation through parametric and barycentric coordinate representations.

Advances in robustness emerged in the early 2000s, notably with the work of Guigue and Devillers [111], who employed robust orientation predicates to handle numerical instability. Their algorithm calculates intersections via determinant-based predicates, circumventing explicit geometric constructions and significantly enhancing robustness in degenerate scenarios.

Parametric-based intersection. In CAD systems, surface geometry is represented by parametric patches (e.g., NURBS surfaces [112]). The computation of raw intersections begins with determining intersection points or curves in the local parameter spaces of the involved patches [113,114]. Subsequently, a topological structuring phase is required to assemble these geometric fragments into a consistent intersection graph, ensuring both topological validity and robustness of the final Boolean operation result [28].

Raw intersection techniques have been specifically developed for parametric and analytic surfaces, where the computation reduces to solving nonlinear algebraic equations [115–117]. Numerical approaches commonly employed in this context include iterative methods such as adaptive subdivision [115,118], validated marching schemes based on interval arithmetic [117], and Newton iterations coupled with topological validation strategies [119]. Alternatively, algebraic methods based on symbolic elimination, such as Bézout or Dixon resultants, attempt to solve the intersection equations in closed form, yielding exact algebraic representations of intersection curves [120,121]. While these approaches provide mathematical exactness, they are computationally demanding and generally feasible only for surfaces of low algebraic degree or for niche applications requiring strict geometric rigor. A detailed discussion of the above algorithms falls within the broader topic of surface-surface intersection (SSI) [122] and is beyond the scope of this survey.

However, the numerical intersection stage typically produces only raw geometric primitives, short curve segments or isolated intersection points, rather than topological entities. These primitives are often fragmented, unordered, or inconsistent across adjacent parametric patches. To support subsequent stages, the fragments must be assembled into topological entities. The assembling process usually begins with adjacency-driven tracing techniques that connect nearby segments into continuous curves, chains, or closed loops across intersected patches [28,29]. Numerical artifacts such as duplicate vertices, degenerate edges, or small curve gaps frequently arise in this stage, due to rounding errors or mismatched parameterizations. These issues are commonly mitigated through geometric cleanup operations, including vertex snapping, tolerance-based clustering, and fragment filtering [38,40].

After cleanup, the intersection results are organized into higher-level structures such as oriented loops, or intersection graphs [14]. In exact-computation frameworks, these structures are constructed using combinatorial arrangements of curve segments together with algebraic predicates, ensuring both topological consistency and robustness [32,34].

In summary, mesh-based intersection methods focus on efficient and parallelizable triangle-triangle testing, while B-Rep-based approaches require a two-stage process involving both accurate computation of raw intersection data and its conversion into valid topological structures. This distinction reflects the intrinsic differences in geometric complexity, precision requirements, and application domains between the two representation paradigms.

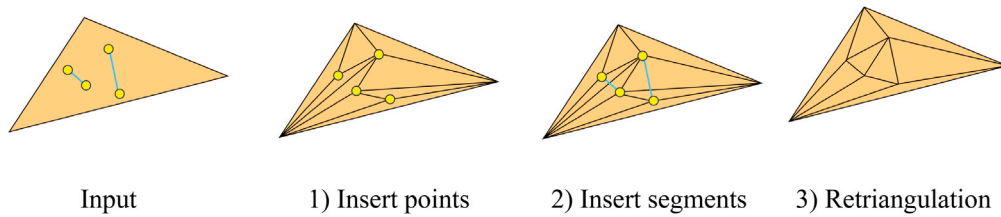


Fig. 8. Local splitting of an intersected triangle, adapted from [70]. Intersection points and segments are inserted, and the resulting region is retriangulated.

3.3. Geometric splitting

After computing intersection elements, edges and faces are split by them. This step is not only a geometric operation but also a topological one, where local topological information is considered, such as adjacency and loop orientation to ensure shape validity.

Geometric splitting strategies depend on the underlying representation. We also distinguish between two dominant categories: mesh-based and B-Rep-based methods. While mesh-based methods focus on discrete topological edits and local triangulation updates, B-Rep-based approaches emphasize precise curve-surface intersections and consistent topological refinement of trimmed parametric patches.

Mesh-based splitting. In mesh-based Boolean operations, geometric splitting is commonly realized as a three-stage process, intersection insertion, segment embedding, polygon retriangulation, illustrated as Fig. 8.

Intersection points insertion. The splitting process begins with identifying and inserting geometric intersection points into the mesh. Early approaches compute these intersections explicitly, solving for triangle-triangle intersection points and introducing new mesh vertices accordingly. A canonical instance is Fujio's pipeline [51], which directly splits edges at computed coordinates.

However, explicit intersection often suffers from numerical instability, especially when dealing with nearly coplanar configurations or degenerate geometries. To address this, later methods adopt symbolic or deferred insertion strategies, where intersections are represented implicitly via predicates or algebraic handles. This approach avoids premature rounding errors and allows consistent handling of repeated intersection queries across mesh edits [70,76].

Segment embedding. After computing intersection vertices, the next step is to embed the corresponding intersection segments into the affected faces. A straightforward strategy inserts segments incrementally: each segment is traced across the mesh, intersected triangles are removed, and the hole is re-tessellated while treating the segment as a constrained edge. When a segment intersects existing constraints, new vertices are introduced to split edges, thereby avoiding T-junctions or overlapping fragments [54,66,70]. A more robust alternative is to process all segments within a face simultaneously. In this arrangement-based approach, the full set of intersection segments is split at crossings and endpoints, mapped into the face's local 2D parametrization, and used to build a planar arrangement. The resulting subdivision is then consistently lifted back to 3D [65,69]. By resolving all intersections in one shot, this strategy removes order dependence and prevents inconsistencies such as T-junctions or zero-length edges. Modern pipelines, such as EMBER, further combine local arrangements with exact integer representations to achieve both robustness and efficiency [76].

Retriangulation. Having embedded intersection segments, the affected faces must be triangulated into valid cells. Early methods simply cut intersected triangles into planar polygons and retriangulated them locally with ear-clipping or template rules [51]. Early schemes, such as Feito et al. [60], inserted additional edge-triangle intersections and applied a two-step local tessellation to remove residual crossings. Later pipelines emphasized mesh quality: adaptive remeshing aligns intersection boundaries within a tolerance and closes gaps

by constrained zippering, as in Adaptive Mesh Booleans [64]. EMBER [76] instead employs a plane-based integer representation with face-local arrangements, avoiding incremental constrained Delaunay triangulation while ensuring exactness. For simple polygons generated by constrained insertions, deterministic linear-time ear-cutting offers predictable triangulations with both robustness and good element quality [70,123].

B-Rep splitting. In B-Rep models, splitting is the process of subdividing faces into smaller regions so that classification can be applied locally. Although the underlying analytic surface is usually preserved, splitting requires both geometric and topological updates: intersection curves must be computed and inserted, parameter-space loops constructed, and new trimmed regions defined.

Early work by Requicha and Voelcker introduced a generate-and-test strategy in the PADL boundary evaluation model [28]. Candidate intersection edges are first generated on the supporting surfaces, often longer than the true boundary. These oversized curves are then trimmed by set-membership tests in the parameter domain, leaving only the valid edge segments. The resulting edges are assembled into closed loops that bound maximal faces, thereby realizing the splitting step. Brisson proposed the cell-tuple representation [29], which provides a general way to encode how new intersection edges subdivide faces and maintain adjacency. When a splitting curve is inserted, the data structure introduces new cells and updates local cyclic order around lower-dimensional entities, ensuring topological consistency of the subdivided model. The splitting principle was made explicit by Satoh and Chiyokura [37]. Their method computes surface-surface intersections, maps them to the parameter domains, and combines them with existing edges to form trimming loops. Each loop subdivides the original face into distinct subregions, which are later classified and merged. The BOOLE system [43] advanced this idea for sculptured solids: intersection curves were robustly computed, checked for closure, and then used to split each surface patch into trimmed components. The essential idea is that intersection curves, when mapped into parameter domains, form trimming loops that subdivide the original surface into smaller regions. These loops correspond to trimmed patches in 3D, as illustrated in Fig. 9.

Further work emphasized robustness in this stage. Tayebi et al. [57] developed an implementation for NURBS surfaces, where intersection curves directly define trimming loops that partition surfaces into accurate trimmed patches. More recently, Urlick et al. [49] proposed a framework for enforcing watertightness in trimmed-surface B-Rep models: instead of leaving trimmed surfaces with tolerance gaps, they reparameterize intersecting patches and update the model so that trimming curves become explicit surface boundaries. This approach produces watertight, tolerance-controlled solids suitable for downstream CAD and CAE applications.

Both mesh- and B-Rep-based methods partition input surfaces into coherent fragments bounded by intersection curves. Mesh-based splitting achieves this by inserting intersection elements and locally retriangulating, thus altering the mesh to fit new boundaries. In contrast, B-Rep splitting preserves analytic surfaces and performs a purely topological partition by redefining closed loops in parameter space. Despite these differences, both approaches yield fragments with well-defined boundaries, ensuring unambiguous classification in subsequent stages.

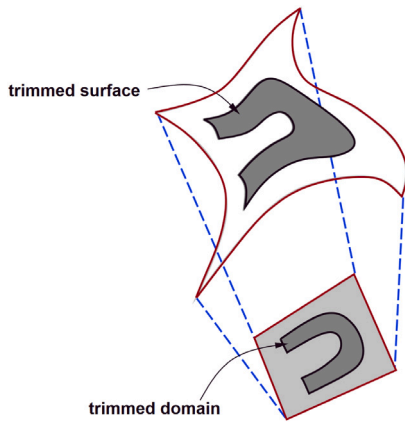


Fig. 9. An example of the trimmed surface. It illustrates a mapping from a trimmed domain in parameter space to a trimmed surface in 3D [43].

3.4. Fragment classification

Classification assigns each geometric fragments produced by the geometric splitting a SMC label (inside, outside, or on) relative to the other shape. This step directly determines which fragments form the final Boolean result.

Two major strategies are commonly used for fragment classification: ray-casting and winding-number. As illustrated in Fig. 10, a 2D example shows how two points are classified: ray casting determines the inside/outside by intersection parity, while the winding number integrates oriented angle contributions.

Ray-casting classification. Point-in-solid classification via ray casting has been a foundational strategy since the earliest B-Rep boundary evaluation methods. In Requicha and Voelcker's formulation, a fixed-direction ray is cast through the query point, and its number of intersections with surface faces is counted to determine inclusion via the odd-even rule [28]. Hoffmann later improved robustness by introducing a cascading ray scheme: when the primary ray encounters topological degeneracies, such as hitting a vertex or grazing an edge, secondary rays in alternative directions are recursively cast to avoid ambiguity [35].

Subsequent systems further enhanced performance and stability. For example, BOOLE caches previously evaluated rays to avoid redundant tests during interactive sessions [43], while Hui and Kan [42] parallelize classification over spatial subdivisions to scale with model complexity.

In polygonal settings, ray-based classification has been adapted to triangle meshes without manifold assumptions. Yongbin et al. [54] extend the ray casting approach to triangle soups by coupling each ray-face test with local edge-orientation heuristics, thereby achieving sublinear query complexity.

Modern interactive Boolean systems refine this idea into a multi-stage evaluation cascade. They first perform fast floating-point ray tests; when the result is ambiguous, due to numerical degeneracy or proximity to features, they fall back to rational arithmetic for exact evaluation. In rare pathological cases where even rational tests become unstable (e.g., near-coplanar configurations), symbolic perturbation is employed to guarantee consistency [70]. This tiered approach retains the efficiency of classical ray casting while the robustness required for industrial-scale models.

Winding-number classification. Winding-number methods provide an alternative to geometric point-in-solid tests by evaluating how many times a surface wraps around a region in space.

A notable discrete realization of this principle was proposed by Feito and Torres [124], who introduced a sign-accumulation method based

on simplicial coverings. In this formulation, each triangle of the mesh forms a tetrahedron with a common reference point (e.g., the origin), and the point-in-solid status is evaluated by summing orientation signs over all tetrahedra. A point is considered inside the solid if the accumulated value exceeds a fixed threshold (typically 1), mirroring the integral interpretation of winding number. Ogayar et al. [52,125] further adapted this scheme for GPU implementation, achieving high performance through parallelized evaluation. Conceptually, this approach is equivalent to a discrete volumetric winding number and inherits its robustness and topological correctness even on complex polyhedral domains.

Smith and Dodgson [53] later applied the discrete winding number to Boolean classification, showing that it changes by ± 1 whenever a traversal crosses an oriented intersection curve on a closed triangle mesh. This enables the inside/outside status of surface fragments to be determined incrementally without performing repeated containment queries.

To improve robustness under coarse tessellations and self-intersections, Jacobson [62] introduced the generalized winding number, a continuous extension based on the summation of signed solid angles subtended by mesh triangles at a query point. This formulation remains stable on imperfect or non-manifold inputs and has since been widely adopted in robust mesh processing frameworks.

Zhou et al. [65] further advanced the method by formulating classification as a discrete propagation process over the surface arrangement defined by all intersection curves. Starting from an exterior cell initialized with a winding vector, the algorithm traverses neighboring regions and updates the k th component of the vector upon crossing a face belonging to solid A_k . This combinatorial viewpoint naturally supports multi-object Booleans and eliminates the need for query-point sampling.

Recent advances have focused on accelerating winding-number evaluation to handle large-scale and non-watertight geometry. Barill et al. [126] introduced a hierarchical summation scheme for triangle soups and point sets, organizing primitives in a Barnes–Hut style tree.¹ This reduces the cost of each query to near-logarithmic complexity, while closely approximating the exact generalized winding number. Building on this hierarchical paradigm, Madan et al. [128] proposed a stochastic Barnes–Hut approximation tailored for GPU execution. By randomly sampling cluster contributions at different depths of the Barnes–Hut tree and applying control variates for variance reduction, their estimator remains unbiased and achieves the same median accuracy up to $9.4\times$ faster than a deterministic GPU Barnes–Hut implementation. These developments significantly broaden the practical applicability of winding-number-based classification, enabling interactive and large-scale processing scenarios.

Ray-based and winding-number-based classification offer complementary strengths. Ray-casting tests point inclusion individually, making it simple and parallel-friendly, but it requires special handling near geometric degeneracies. Winding-number methods, by contrast, compute region-level topological invariants, enabling path-independent and globally consistent classification at the cost of increased complexity and preprocessing.

3.5. Topological reconstruction

In mesh-based Boolean pipelines, computation typically ends after valid surface fragments are identified and filtered by classification. Additional steps, such as repairing non-manifold edges, filling small gaps, or merging disconnected components, may be required to restore watertightness and geometric validity, and to ensure compatibility with downstream meshing, simulation, and fabrication pipelines. However,

¹ The Barnes–Hut method is a hierarchical approximation algorithm originally developed for fast N-body simulation using a tree structure [127].

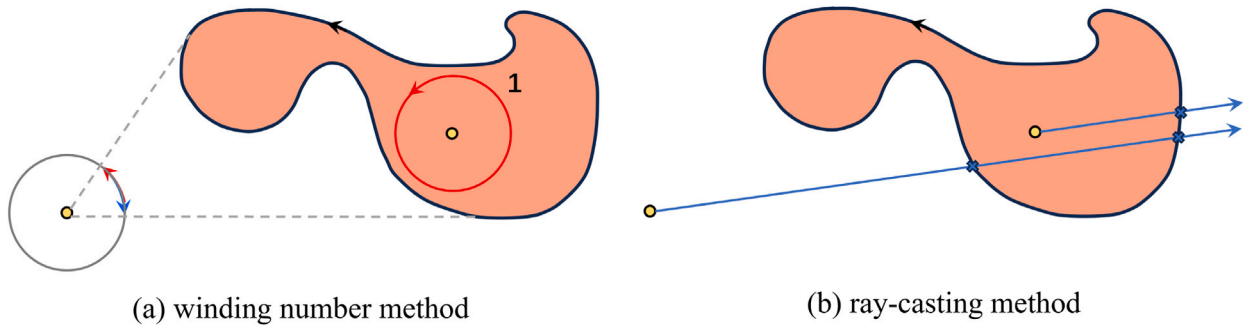


Fig. 10. Comparison of two point-in-solid classification strategies. (a) Winding-number method: the oriented angles of boundary segments accumulate around the query point. (b) Ray-casting method: inside/outside is determined by counting ray-curve intersections.

these operations are generally considered mesh repairs [129], and a detailed discussion of them is beyond the scope of this survey. In contrast, B-Rep systems explicitly reconstruct hierarchical topological entities, including shells, faces, and bodies, to preserve solid-modeling semantics, ensure kernel consistency, and maintain interoperability across CAD systems. Consequently, topological reconstruction in B-Rep frameworks is an essential and integral part of the modeling process.

In B-Rep-based systems, the reconstruction stage explicitly rebuilds the hierarchical shell-body structure. After trimming surface patches along intersection curves, consistent fragments are grouped into closed, oriented shells. Systems such as BOOLE and Satoh's surface-based approach demonstrated watertight shell construction by robustly matching trims and enforcing closure checks [37,43]. Once shells are assembled, they are organized into bodies according to spatial containment: outer shells define external boundaries and inner shells cavities. Masuda formalized this process with cell-complex operators for non-manifold models [39], while Gursoz et al. [38] extended reconstruction to handle shared geometry in non-manifold B-Reps. Paoluzzi's algebraic frameworks [34,50] further abstract this process as the manipulation of cell complexes.

In contrast to mesh-based approaches, B-Rep systems must reconstruct the complete topological structure of the solid. This includes identifying closed shells from classified surface fragments and organizing them into bodies based on spatial containment. The result adheres to modeling kernel standards, ensuring topological validity and semantic correctness for downstream CAD applications.

4. Boolean operations for other representations

Although Boolean algorithms based on explicit representations are widely applied, their robustness problems arise largely from the explicit representations themselves rather than from the Boolean operations, and many of these issues disappear when other representations are used. This section discusses Boolean operations for five common representations, including CSG, BSP, volumetric, implicit, and hybrid representations, which differ from those described in the pipeline above. In addition, line representation (L-Rep) [130] has been proposed as another representation, but due to its limited adoption and relatively few follow-up works, it is not discussed in detail here.

4.1. CSG representations

Background. CSG [20], developed in the 1970s, was one of the earliest formal paradigms for solid modeling. It represents shapes as Boolean combinations of primitive solids, providing a compact and hierarchical description that remains useful in procedural and parametric modeling. While CSG played an important role in early CAD systems and remains valuable in visualization [88,89], and collision detection [131], modern industrial CAD kernels such as ACIS, Parasolid, and OpenCASCADE are predominantly based on B-Reps. In contemporary CAD workflows, CSG

typically serves as a high-level feature or construction history rather than as the underlying geometric evaluation model.

In CSG, a solid model is expressed as a CSG tree, a rooted binary tree where leaf nodes represent basic primitives (e.g., cube, sphere, cylinder) and internal nodes represent Boolean operators such as union, intersection, and difference. Extended CSG trees [83], proposed for systems based on implicit primitives, allow internal nodes to represent Boolean, Warping, and Blending operations. Recent algebraic approaches [12] further interpret CSG expressions as elements of a finite Boolean algebra, enabling their evaluation through sparse matrix operations in a linear algebraic framework.

Here, we focus on evaluating CSG models of three-dimensional solids where primitives are full 3D volumes. This differs from plane-based CSG or BSP approaches which work on half-spaces or planar partitions. While a number of works address related topics such as CSG visualization [88,89], or differentiable modeling [132], we concentrate on the evaluation of CSG expressions into explicit representation.

Methods. The most basic approach to CSG evaluation is recursive tree traversal. A CSG model is represented as a rooted binary tree, with leaf nodes as primitives and internal nodes as Boolean operators. Evaluation proceeds by traversing the tree and computing geometry at each internal node by applying the corresponding Boolean operation to its children.

Early studies proposed optimizations to minimize redundant computation and reduce the number of intermediate results. Bronsvort [87] introduced bottom-up evaluation, partial back-face elimination, and active-face table reordering to reduce Boolean evaluations in scan-line algorithms, achieving a reported 65%–80% reduction in evaluation time. These techniques greatly improved interactive performance but were limited to planar-face objects and scan-line frameworks.

Later works addressed not only speed but also robustness and numerical reliability. BOOLE [43] combined computational geometry, numerical linear algebra, and symbolic computation to robustly evaluate boundary representations of sculptured solids, using exact intersection curves and topological checks for consistency. ESOLID [46] further improved reliability by adopting exact arithmetic, floating-point filters, and lazy evaluation techniques, thereby eliminating numerical errors and guaranteeing consistent results. These methods can handle industrial-scale models but incur high computational and memory costs.

An influential work avoids the sequential binary merging inherent in CSG trees. QuickCSG [90] uses a vertex-centric, output-sensitive approach to solve n-ary Boolean combinations directly. Instead of building intermediate results for each node, it enumerates candidate vertices from primitive intersections, classifies them with the global Boolean function, and retains only those that contribute to the final boundary. The method integrates classification with spatial subdivision by embedding all input solids into a single KD-tree. It prunes large empty regions and reduces complexity to depend on the output size rather than the number of input primitives. A work-stealing scheduler

yields near-linear parallel speedup on multicore CPUs. The algorithm generates watertight, redundancy-free results suitable for large-scale modeling tasks. However, its robustness may degrade for degenerate inputs, and deep KD-tree stitching can become a bottleneck in large scenes.

Recently, researchers proposed algebraic formulations of CSG evaluation that hide low-level geometric computations. Paoluzzi et al. [12] expressed CSG expressions as elements of a finite Boolean algebra, with atoms generated by space decomposition. The resulting Boolean vectors are evaluated with native bitwise operators and then mapped back to geometry using Linear Algebraic Representation (LAR) and sparse matrix multiplications. This approach guarantees logical consistency, removes numerical instabilities, and scales well to large models, though its current implementation supports only piecewise-linear solids.

Discussion. These approaches illustrate the evolution of CSG evaluation from simple recursive tree traversal to robust, scalable, and algebraically principled methods. Classical tree evaluation is clear in concept but inefficient for large models because of excessive intermediate results. Early optimizations improved interactive performance but applied only to specific representations. BOOLE [43] and ESOLID [46] brought numerical robustness and topological consistency at the cost of high computational overhead. QuickCSG [90] broke away from binary tree merging, offering output-sensitive, parallel evaluation well-suited for large-scale and multi-object scenarios, though less robust under degeneracies. Recent algebraic formulations ensure logical consistency and scalability but remain limited to piecewise-linear solids. Overall, these methods reveal a trade-off among robustness, efficiency, and generality, motivating hybrid and parallel approaches to balance correctness and performance.

4.2. BSP representations

Background. Binary Space Partitioning (BSP) is a plane-based formulation of CSG, where solids are expressed as Boolean combinations of half-spaces. It recursively subdivides space with planes, producing a binary tree whose leaves represent convex regions classified as inside or outside the solid. Giraud [23] introduced *presque* half-planes, a formal scheme that preserves closure and uniqueness of Boolean expressions while supporting infinite regions and degenerate configurations. Thibault and Naylor [72] extended the concept to 3D polyhedra and showed that solids of any dimension can be converted into BSP trees through recursive plane insertions. The resulting representation was dimension-independent, robust, and able to encode semi-infinite solids, enabling exact Boolean algebra over geometric primitives. Naylor et al. [73] later formalized BSP tree merging, providing an elegant algorithm to align space partitions and perform Boolean algebra cell by cell. These studies established BSP as a mathematically well-defined framework for performing set operations over polyhedral, primarily planar-faced geometry.

Although historically significant, BSP has seen limited adoption in modern CAD kernels. Its reliance on planar partitions makes it suitable mainly for polyhedral or piecewise-linear models, and it is used today primarily in graphics and spatial partitioning contexts rather than as a core geometric representation in CAD systems.

Methods. In a typical BSP-based Boolean pipeline, the first step is to construct BSP trees for the input solids. Starting from a polygonal boundary representation, faces are iteratively chosen as splitting planes and used to subdivide space recursively. Faces intersecting a plane are split into fragments and distributed to subspaces until all cells are convex. Each leaf is labeled as inside or outside the solid. Boolean operations are performed by merging the BSP trees of two solids, synchronizing their partitions through recursive plane insertion and combining leaf classifications according to the desired operation (union, intersection, or difference). The result is a new BSP tree encoding the Boolean combination. A consistent boundary representation

is reconstructed by collecting faces that separate inside from outside regions and discarding redundant internal faces.

Subsequent research has emphasized robustness and performance. Bernstein and Fussell [74] introduced a minimal geometric kernel based on only four predicates, handling all degeneracies robustly while running up to 28× faster than CGAL. Nehring-Wirxel et al. [75] proposed octree-embedded BSPs for iterated CSG. Their approach localizes modifications and controls tree complexity even after many operations. They also used 256-bit integer arithmetic for unconditional robustness, achieving up to 2.5 million plane-mesh cuts per second per core. EMBER [76] further improved scalability by combining adaptive kd-tree subdivision, local BSP computations, and winding-number-based classification. This enables exact, parallelizable Booleans for general polygon meshes, including non-manifold and degenerate inputs.

Discussion. BSP-based Boolean operations have several key advantages. First, they handle degeneracies such as tangential contacts and disconnected components robustly, due to their space-partitioning nature. Second, they support topologically complex solids with arbitrary genus and partial boundaries, making them suitable for general-purpose modeling. Third, they maintain regularity by design, producing closed sets via topological closure over open cells. However, the non-uniqueness of BSP construction, especially hyperplane selection, can cause tree imbalance and raise computational cost. Moreover, the representation lacks explicit boundary parametrization, requiring extra effort to extract surface meshes for rendering or simulation. Despite these limits, BSP trees remain a strong representation when exact set membership and logical composability are prioritized over compactness or surface continuity.

4.3. Volumetric representations

Background. Boolean operations based on explicit boundary representations (e.g., B-Rep, triangle meshes) are often sensitive to degeneracies, non-manifold features, and ill-defined input. These issues motivate using volumetric representations, where geometry is embedded in regular or hierarchical grids such as octrees or polytrees. Octrees and their variants perform set operations by classifying cells (e.g., full, empty, boundary-intersecting) and recursively subdividing space to resolve local ambiguities. This method generalizes to non-boundary-conforming models, allowing operations on scanned data, sculpted surfaces, and non-watertight inputs. Extensions such as non-minimal division octrees and BSP-embedded volumetric grids further improve accuracy and memory efficiency. Hierarchical sparse voxel data structures such as OpenVDB [133], originally developed for feature film volumetric effects, also support efficient volumetric operations and have been adopted in several modeling and simulation contexts.

Methods. Early work by Ayala et al. [77] introduced edge-classified octrees, distinguishing nodes as interior, exterior, or boundary-intersecting. This reduced storage requirements and enabled exact boundary reconstruction for polyhedra. Polytrees [78] represent spatial regions by attaching explicit polygonal descriptions to each cell, which often leads to redundancy and inconsistency across cell boundaries. The method was later extended to free-form geometry [79], representing sculptured surfaces of biquadratic patches in an extended octree model. Specialized intersection tests (patch-node, curve-face, line-patch) classify cells as face, edge, or vertex nodes. This enables Boolean operations between solids and sculptured surfaces and supports conversion back to boundary representations when needed.

Building on these foundations, Brunet and Navazo [80] refined extended octrees for polyhedral solids. By systematizing face, edge, and vertex terminal nodes, their model achieved an exact polyhedral representation with lower storage cost, while preserving the linear-time Boolean properties of classical octrees.

Adams and Dutré [81] proposed a surfel-based volumetric model, in which oriented surface elements (surfels) are rasterized into a voxel

grid. Boolean operations are performed using depth-buffered rendering passes, comparing input objects slice by slice along viewing directions. This approach sacrifices topological precision for interactive performance and is especially effective for point-based geometry or scanned datasets.

Building on sampling-based volumetric operations, Wang [82] proposed an LDI-based partial mesh reconstruction approach that performs Boolean operations by comparing depth samples along structured view rays. Instead of fully remeshing the result, the method preserves original surfaces outside the intersection regions and reconstructs only boundary neighborhoods using trimmed adaptive contouring, yielding watertight results with lower computational cost. This hybrid strategy combines the robustness of volumetric classification with the fidelity of mesh preservation, making it suitable for large-scale polyhedral models where full voxelization would be prohibitively expensive.

Discussion. Volumetric methods offer strong guarantees in terms of robustness and applicability to noisy or underspecified data. They reduce Boolean operations to hierarchical occupancy evaluations and avoid floating-point predicates, inherently mitigating numerical instability. The uniform spatial structure also supports parallelization and out-of-core computation.

However, these benefits come with trade-offs. Resolution must balance geometric fidelity and memory use: high-resolution grids grow prohibitively large, while coarse grids can cause aliasing and connectivity artifacts due to limited sampling. Techniques like extended octrees address this by refining only where needed.

Another limitation is output expressiveness. Volumetric results often need conversion back to surface-based formats (e.g., B-Rep or triangle meshes) for downstream tasks. This conversion is challenging and may cause ambiguity or detail loss. Several works [56,82,93] propose hybrid representations or pipeline integration with boundary models to mitigate this issue.

In summary, volumetric representations provide a principled and robust foundation for Boolean operations, particularly for complex, irregular, or non-solid inputs. Although once considered approximate or low-resolution, modern hierarchical techniques have greatly enhanced their precision and computational feasibility.

4.4. Implicit representations

Background. Implicit representations define solids using continuous scalar fields [25], such as signed distance functions (SDFs) [134], soft occupancy functions [135], or general algebraic expressions [136]. These fields encode the shape as the zero-level set of a function $f(x) = 0$, with interior and exterior regions determined by the function's sign. Any loss of fidelity arises only from numerical sampling or reconstruction rather than from the representation itself. Boolean operations can be expressed using min/max functions. Compared to explicit boundary representations, implicit models offer greater topological flexibility, naturally support non-manifold configurations, and avoid explicit boundary intersection handling. As a result, they are often regarded as robust in scenarios involving frequent topology changes and are well suited to parallel evaluation on modern hardware. Such properties have motivated their adoption in both research systems and industrial tools, including nTop (nTopology) [137], Adobe Substance [138], Womp [139], and MagicaCSG [140].

Methods. Early methods for Boolean operations on implicit surfaces employed p -norms [136], R-functions [24], soft min/max operators [83], or polynomial smoothing [85]. These approaches supported smooth transitions and were popular in computer animation and sculpting tools. Qin et al. [84] introduced an efficient cut-and-paste modeling method that converts polygonal meshes into point clouds and applies radial basis function (RBF) interpolation to compute implicit fields.

Recently, Liu et al. [9] proposed a unified differentiable Boolean operator for implicit representations using fuzzy logic principles, as Fig.

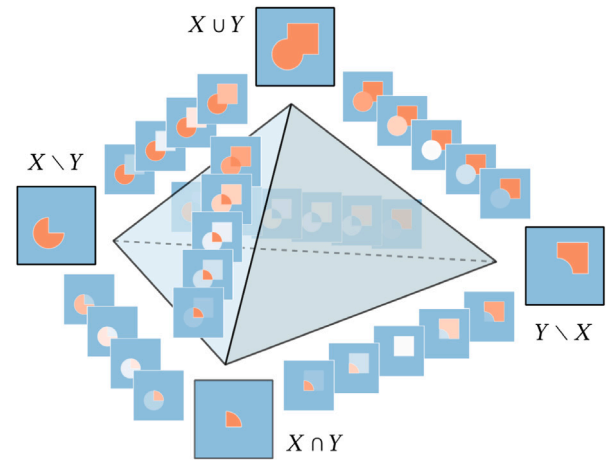


Fig. 11. Liu et al. [9] use barycentric interpolation to produce monotonic interpolation between different Boolean operators.

11. They replaced discrete min/max functions with tetrahedral interpolation of fuzzy set operators, enabling continuous interpolation between union, intersection, and subtraction. This approach yields scalar fields differentiable with respect to both shape parameters and Boolean operation types, enabling gradient-based optimization for inverse CSG. This flexibility supports modeling both smooth organic forms and sharp mechanical structures in a single framework.

Discussion. Implicit representations provide a robust framework for Boolean operations by encoding shapes as continuous scalar fields. This approach naturally handles topological changes and geometric degeneracies. Boolean operators reduce to simple function compositions, producing watertight results without explicit intersection handling or re-meshing. These properties make them effective for volumetric modeling, shape blending, and deformation.

Nonetheless, these benefits involve challenges. Scalar field evaluation and mesh extraction are computationally expensive at high resolutions. Compositions can introduce artifacts, including unintended blending or loss of sharp features. Handcrafted functions often fail to capture detailed or heterogeneous structures, which limits adoption in CAD.

Future directions emphasize adaptive and multi-resolution sampling to reduce computational load. Hybrid methods that combine implicit fields with meshes or B-Reps aim to balance robustness and efficiency. Neural implicit approaches aim to improve expressiveness, generalization, and differentiability for optimization tasks.

4.5. Hybrid representations

Background. Hybrid representations have recently emerged as a promising approach, combining the complementary strengths of explicit and implicit models. The main motivation is to address robustness issues and performance bottlenecks in conventional B-Rep Boolean operations, especially when processing heterogeneous input such as CAD models, polygonal meshes, and scanned point clouds.

Methodologically, research in this area falls into two complementary directions. The first direction focuses on hybrid operations across representations. It enables Boolean computation between heterogeneous inputs by converting them into a common computational domain or by switching representation locally near intersections. The second direction explores bijection-based hybrid acceleration. It builds a one-to-one correspondence between representations so that costly operations can be performed in an efficient domain and then mapped back exactly to the original model.

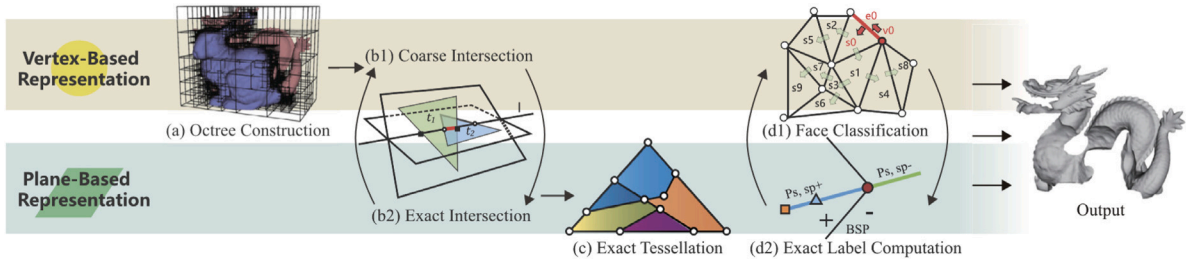


Fig. 12. An overview of the hybrid Boolean pipeline by Sheng et al. [27].

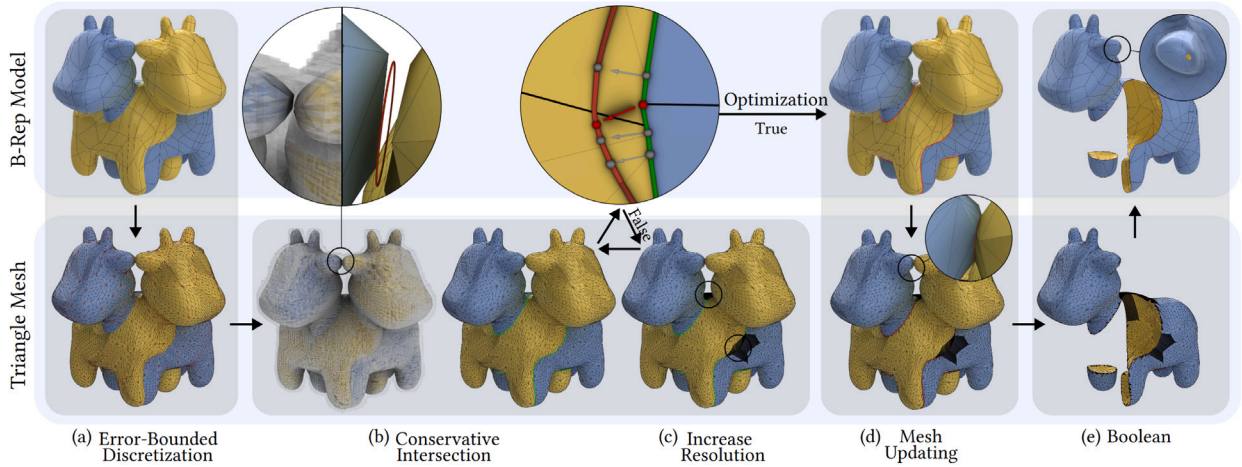


Fig. 13. Overview of the hybrid B-Rep/mesh Boolean framework of Yang et al. [5]. The method establishes a bijection between the input B-Rep models and a triangle-mesh discretization.

Methods.

Hybrid across representations This line of work aims to make Boolean operations robust and compatible across heterogeneous inputs. Fougerolle et al. [91] proposed a framework using R-functions to merge parametric primitives into differentiable implicit equations, followed by precise polygonization. Parameter-space subdivision refined intersection regions, yielding robust, watertight results across parametric and implicit primitives. Yang and Qian [92] tackled direct Boolean intersections between designed geometry (objects bounded by NURBS and polygonal surfaces) and scanned geometry (objects represented by point cloud data) without explicit reconstruction. Their method combined curvature-adaptive marching, Gauss–Newton refinement, and adaptive quadtree subdivision to locate intersections accurately and generate trimmed surface patches. These approaches emphasize conversion to a common domain or local representation switching, improving interoperability and numerical stability. The main limitation lies in the overhead of representation conversion and the dependence on discretization or sampling resolution to achieve sufficient accuracy.

Bijection-based acceleration The second line of research focuses on efficiency and scalability by establishing and maintaining a bijective correspondence between different representations. Pavić et al. [93] combined volumetric and surface-based schemes using adaptive octrees, exploiting the robustness of volumetric methods for intersection detection while preserving surface-level accuracy for final reconstruction. Upreti and Subbarayan [94] proposed signed algebraic level sets for NURBS surfaces. They used algebraic implicitization and R-function composition to perform implicit Boolean operations and non-iterative point-in-membership classification without explicit curve–curve intersection computation. This approach preserves exact CAD geometry and supports seamless CAD–CAE integration. Sheng et al. [27] presented a hybrid framework that combines vertex-based (V-Reps) for coarse detection and adjacency queries with plane-based (P-Reps) for exact

geometric predicates, as illustrated in Fig. 12. Delayed tessellation and local BSP trees enabled efficient multi-input Boolean evaluation while maintaining robustness. Most recently, Yang et al. [5] constructed a provably bijective mapping between exact B-Rep models and controlled-accuracy triangle meshes, as illustrated in Fig. 13. Boolean operations are executed entirely in the mesh domain, and the resulting intersections are mapped back to the B-Rep model, achieving watertightness and reporting 2 – 20× speedups compared with traditional B-Rep kernels.

Discussion. Hybrid representations offer an attractive compromise between the high accuracy of explicit B-Rep modeling and the robustness of implicit approaches. They enable Boolean operations over heterogeneous input types, avoid solving explicit curve–curve intersection equations, and support scalable parallel implementations such as mesh-based and octree-based pipelines. Experiments show improved robustness in degenerate or near-tangent cases while preserving high-quality, watertight outputs [5].

Nevertheless, challenges remain. Representation conversion and bijection construction are often computationally expensive, and accuracy is sensitive to discretization resolution and mapping quality. Memory overhead may be significant for large models and extremely fine features can still cause robustness issues.

Future research is likely to explore adaptive hybridization, where the choice of representation is made dynamically based on local geometric complexity; learning-based mappings, where neural networks predict intersection locations or approximate implicitizations to accelerate computation; and fully unified pipelines that combine cross-representation operations with bijective mappings for robust, watertight, and potentially real-time Boolean computation on large CAD assemblies.

5. Challenges

Boolean operations are a fundamental part of 3D geometric modeling. They enable the construction of complex models using set operations. However, their implementation continues to face persistent challenges. These challenges can be grouped into three aspects: **robustness and numerical stability**, which ensure correct and consistent geometric and topological outputs under complex inputs and finite-precision algorithms; **efficiency and scalability**, which determine the feasibility of applying the operations in large-scale or interactive applications; and **extensibility and generalization**, which concern algorithmic flexibility to support diverse input representations and enriched semantics of operations.

Although these aspects are interrelated in practice, this section presents each challenge in turn, reviewing classical obstacles, representative strategies, and recent advances. By analyzing them separately, we aim to clarify the technical difficulties and solutions that support reliable and efficient Boolean operations across diverse geometric modeling scenarios.

5.1. Robustness and numerical stability

Robustness and numerical stability remain major challenges for Boolean operations. These issues mainly arise from finite-precision arithmetic, near-degenerate configurations such as coplanarity and near-tangency, and subtle topological artifacts such as cracks, non-manifold connections, and degenerate elements (e.g., zero-area faces or zero-volume cells). To address these issues, researchers have proposed three main approaches: numerical-model-level approach, which defines how numbers are represented and manipulated; representation-level approach, which prevents problematic numerical conditions by using numerically favorable data representations. While numerical-model-level approaches provide the theoretical foundation for correctness, they often face trade-offs between exactness and efficiency. Representation-level approaches reduce fragile cases, but it typically replaces numerical brittleness with other limitations, such as approximation bias, resolution dependence, or higher computational cost, rather than resolving the underlying challenge. Since imperfect and noisy inputs are common, the core challenge remains to develop intrinsically robust algorithms, with the most reliable solutions usually achieved through co-design of numerical models, representations and algorithmic strategies.

5.1.1. Symptoms and causes

Robustness has long been a major challenge in Boolean operations for 3D modeling, since numerical errors and degenerate configurations can propagate through the pipeline and cause global failures [18]. A key difficulty arises from the limits of floating-point arithmetic: intersection points are often computed with small but significant errors, and even minor rounding can accumulate to produce inconsistent results in later stages. These instabilities are especially harmful for geometric predicates, where a wrong decision about whether a point is inside, outside, or exactly on a surface can alter the result. The problem is worsened by degenerate or boundary cases, such as coplanar or overlapping faces, zero-length edges, or multiple intersections around a single vertex. Such configurations frequently introduce inconsistencies in the 2D arrangements of split faces, which in turn make the subsequent 3D topological reconstruction difficult and may lead to ambiguous results. Conversely, when the 2D boundaries are compliant, the reconstruction step can be carried out exactly.

In practice, robustness issues are amplified by the quality of the input data. Models from CAD, simulation, or 3D scanning pipelines often contain non-manifold structures, small gaps, holes, or noisy measurements, which deviate from the idealized assumption of watertight polyhedral geometry. Fig. 14 schematically illustrates common input types that are susceptible to robustness failures in Boolean operations.

Such imperfections destabilize intersection computation and frustrate reliable classification. These challenges worsen in large-scale or high-resolution models, where intersections grow rapidly and local errors cascade into global defects such as cracks, missing regions, or invalid topologies in the output. Along the canonical Boolean pipeline, robustness problems appear at the most stages: intersection computation suffers from unstable numerical solutions; geometric splitting produces redundant or degenerate pieces; classification is sensitive to ambiguous in/out/on predicates. These causes explain why robustness remains central and unresolved despite decades of research.

5.1.2. Numerical-models-level approach

Robustness depends not only on algorithm design and data structure, but also on the numerical model. This model defines how values are compared and how geometric relationships, such as equality, inclusion, and validity, are judged when limited precision causes small errors. Different strategies are used: exact arithmetic avoids rounding errors but is slow, tolerance-based methods allow small deviations to improve efficiency, and hybrid approaches combine both for better balance. While data structures store geometric and topological information and algorithms organize operations, the numerical model decides how results are interpreted under numerical errors. Understanding this layer helps distinguish failures caused by numerical precision from those caused by algorithms or data structures, leading to fairer comparisons and more reliable geometric processing.

Exact numerical models. Floating-point arithmetic is the standard in most geometric modeling systems, but it introduces unavoidable rounding errors. These small errors can accumulate and cause inconsistencies in fundamental predicates such as orientation tests or point-in-polygon checks [141]. As a result, models may exhibit topological anomalies, such as gaps, overlaps, or misclassified regions, which undermine robustness [142,143]. Exact numerical models address this issue by replacing approximate floating-point evaluations with mathematically precise computations. The objective is to ensure that every predicate and construction yields results consistent with the underlying mathematical definitions, thereby eliminating failures caused by numerical uncertainty [18].

Exact numerical models are implemented through several techniques. A common approach is rational arithmetic, where numbers are stored as integer ratios. More general methods use arbitrary precision libraries such as GMP [144] or MPFR [145] to represent real numbers with effectively unlimited accuracy. Symbolic evaluation techniques, including expression trees or directed acyclic graphs, are often combined with lazy evaluation, postponing exact computation until needed, to control performance costs [146]. Within this paradigm, the Exact Geometric Computation framework has become an influential principle, prescribing that predicates must always be computed exactly [147]. Libraries such as CGAL provide exact kernels that implement this idea, offering mathematically correct predicates and constructions while balancing efficiency through adaptive evaluation [148].

Despite their reliability, exact numerical models face significant drawbacks. Computations with arbitrary precision are orders of magnitude slower than floating-point arithmetic and may demand large amounts of memory. Engineering such systems also adds complexity, since adaptive evaluation strategies and specialized kernels must be carefully integrated. Consequently, exact models are rarely practical for large-scale modeling, interactive applications, or real-time systems. Their strength lies in guaranteeing correctness, but the cost makes them best suited for contexts where robustness is more critical than performance.

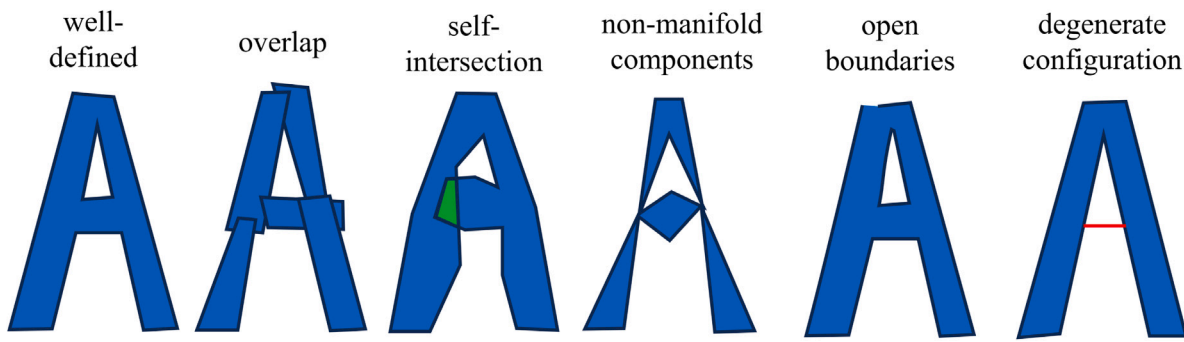


Fig. 14. Representative geometric and topological defects in input models that frequently lead to robustness issues in Boolean operations.
Source: Adapted from [65].

Arithmetic filtering techniques. Geometric modeling relies on numerical computations such as orientation tests, intersection checks, and distance evaluations. These operations are sensitive to floating-point errors, especially in near-degenerate configurations. Using exact arithmetic throughout ensures correctness but is prohibitively slow for large-scale modeling tasks. Arithmetic filtering addresses this trade-off by combining efficiency with robustness [149]. The central challenge is to maintain the speed of floating-point arithmetic while guaranteeing correct results in critical cases.

Arithmetic filtering follows a two-stage strategy. A computation is first attempted with standard floating-point arithmetic. Along with the computed result, an error bound is estimated to test whether the sign or value is numerically reliable. If the outcome is far from uncertainty, the result is accepted. If the computation is close to a threshold or potentially unstable, the algorithm falls back to a more expensive exact arithmetic evaluation [149,150]. This adaptive scheme minimizes unnecessary use of exact computation while preserving correctness. Several techniques support this process, including adaptive precision arithmetic, interval arithmetic, and mixed-precision strategies [151,152]. Widely used frameworks, such as CGAL's filtered kernels and the Exact Geometric Computation paradigm [8,153], provide reusable implementations of these methods for predicates and constructions.

Despite their practical success, arithmetic filters have notable drawbacks. Designing tight error bounds for complex predicates is mathematically demanding and often problem-specific. In highly degenerate or ill-conditioned inputs, many operations may fall back to exact arithmetic, leading to significant performance costs. Furthermore, while the expected case is efficient, the worst-case complexity remains dominated by exact computations. As a result, arithmetic filtering provides a pragmatic compromise but cannot fully eliminate the robustness–efficiency tension in geometric modeling.

Tolerance-based numerical models. Numerical errors are unavoidable in geometric modeling due to the use of floating-point arithmetic and the presence of degenerate configurations. Exact arithmetic can avoid such issues, but it is computationally expensive. To balance robustness and efficiency, tolerance-based numerical models introduce small zones of uncertainty [154,155]. Entities are treated as equal or coincident if they fall within a predefined tolerance.

A common method is ϵ -geometry [156,157], where each point or curve is considered valid within a tolerance neighborhood, allowing small errors to be absorbed without failure. Snap rounding and related grid snapping techniques map coordinates to nearby grid points, eliminating inconsistencies caused by close or nearly coincident entities [158]. These methods ensure that geometric operations produce results that align with a consistent discrete structure. Another family of techniques uses interval arithmetic or affine arithmetic [159], in which computations propagate bounds that capture all possible error values. This bounding process prevents uncontrolled error growth and helps guarantee that results remain valid even in the presence

of uncertainty. These approaches improve robustness while retaining reasonable efficiency.

Despite their practicality, tolerance-based methods face notable drawbacks. The results are highly sensitive to the chosen tolerance values: too small may fail to absorb floating-point noise, while too large may collapse distinct features. Excessive tolerances can also introduce topological inconsistencies, such as overlapping or missing edges in reconstructed models.

5.1.3. Representation-level approach

Representation-level approaches focus on the way geometric models are represented and stored to enhance robustness. These methods aim to improve accuracy and stability by carefully choosing mathematical structures that mitigate numerical errors and geometrical inconsistencies. Following paragraphs explore various representations and their effectiveness in improving geometric modeling across diverse applications.

Nef polyhedra. Nef polyhedra provide a representation that is closed under Boolean set operations and topological operations, making them well suited for exact and robust geometric computation, especially in the presence of degeneracies and non-manifold configurations [16,17]. In implementations such as CGAL's `Nef_polyhedron_3`, the polyhedron is represented as a Selective Nef Complex (SNC), which consists of vertices, a sphere map for each vertex, edges, facets, volumes, selection marks, and their incidence relations, as illustrated in Fig. 15. A sphere map captures the local neighborhood of a vertex by intersecting a small sphere centered at the vertex with the local polyhedral structure, yielding a subdivision on the sphere that encodes the incidences of adjacent cells. The SNC combines these local sphere maps with the global incidence structure, enabling traversal and Boolean operations [17]. Originally formalized by Bieri and Nef, these representations guarantee numerical and topological correctness through closed-form combinational Boolean algebra [32,34]. Despite their rigorous correctness guarantees, their complexity and computational overhead restrict widespread adoption to specialized, verification-critical applications where robustness and correctness override performance concerns.

BSP representations. BSP representations provide a robust computational framework for Boolean operations by recursively subdividing space into a hierarchy of convex regions using exact plane equations. Unlike surface-based approaches that rely on floating-point intersection tests, BSP operates on algebraic half-space classifications, thereby avoiding ambiguities caused by nearly coplanar or tangential configurations. The explicit representation of spatial relationships within the BSP tree ensures topological consistency and precise region labeling, even under numerical perturbations.

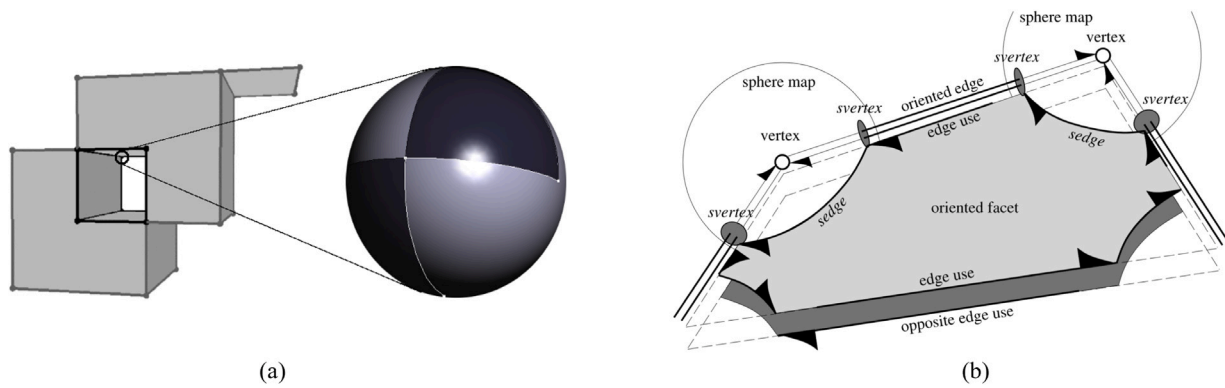


Fig. 15. Data structures used in CGAL's implementation of Nef Polyhedra [16,17]: (a) a sphere map; (b) a SNC.

Implicit representations. Implicit representations describe geometry as continuous scalar fields rather than discrete surface elements, thereby eliminating the need for explicit face-face intersection tests, which the primary source of numerical instabilities in traditional Boolean operations. Their continuous formulation inherently tolerates geometric degeneracies and floating-point imprecision, as small perturbations in input only induce smooth variations in the scalar field. Moreover, smooth blending mechanisms such as R-functions preserve differentiability and ensure stable behavior even under nearly tangential or coincident intersections [83], providing a mathematically robust alternative to explicit boundary evaluation.

5.1.4. Limitations and discussion

Despite decades of research, achieving robustness in Boolean operations remains an unresolved challenge. The root cause lies in the inherent limitations of floating-point arithmetic: geometric predicates based on finite precision and heuristic tolerances cannot guarantee correctness in degenerate configurations such as coplanar overlaps or near-tangent contacts. Recent advances partially address this issue through filtered or lazy exact strategies, which dynamically invoke exact arithmetic only when numerical uncertainty is detected, thereby balancing efficiency and formal correctness.

Beyond arithmetic precision, robustness is further challenged by the heterogeneity of geometric representations, where conversion and approximation errors introduce subtle inconsistencies. Moreover, tolerance specifications remain implementation-dependent. Modeling kernels adopt distinct numerical units and healing strategies, making reproducibility across platforms difficult.

Promising directions include the development of adaptive numerical models that selectively increase precision in numerically sensitive regions, and representation-independent Boolean frameworks that maintain consistent topological validity across multiple model types.

5.2. Efficiency and scalability

The increasing complexity and size of 3D models in CAD, digital fabrication, and graphics require Boolean operations that are robust, efficient, and scalable. Early methods emphasized topological correctness using exact arithmetic. In contrast, modern workflows with high-resolution meshes or real-time editing demand near-interactive performance across varied datasets [70].

The performance of Boolean operations depends on several factors: the cost of intersection detection, the overhead of fragmentation and classification, and the fallback to exact computation for robustness. Design choices, such as spatial data structures, numerical precision, and classification strategies, directly affect runtime and scalability.

This subsection highlights key efficiency bottlenecks and reviews representative optimization strategies in two domains: data structure optimization and parallel or GPU-based acceleration.

5.2.1. Symptoms and causes

Boolean operations often suffer from performance degradation when applied to large-scale or topologically complex 3D models. While small meshes can be processed almost instantaneously, industrial-scale assemblies with millions of triangles may incur delays of several seconds or more. This inefficiency becomes particularly problematic in interactive applications such as CAD editing or digital fabrication previews, where real-time responsiveness is crucial. Moreover, even medium-scale models with intricate topologies or near-degenerate features can trigger worst-case behaviors, leading to memory bloat, unpredictable latency, or outright failure [70,76].

The root causes of these bottlenecks span multiple stages of the Boolean pipeline, with the intersection computation and classification phases contributing the majority of the computational cost. Intersection detection dominates runtime due to the quadratic growth of candidate triangle pairs, particularly in dense or near-coplanar configurations. Although spatial acceleration structures (e.g., KD-trees, BSPs) help prune unnecessary checks, insufficient filtering often results in redundant intersection tests [90]. Further inefficiency arises when exact arithmetic must be invoked to handle degeneracies, ensuring robustness but imposing heavy computational overhead [69,160].

On the other hand, classification becomes a second major bottleneck as it involves evaluating the spatial relationship of numerous fragments and regions generated during intersection. Sliver or near-coplanar faces may inflate memory usage and complicate logical region tests [70,76]. Although localized schemes such as QuickCSG or adaptive octrees mitigate some costs by restricting computation to relevant subspaces [90], maintaining consistent geometric and topological states remains expensive in general cases.

5.2.2. Data structure optimization

Efficient spatial data structures lie at the core of high-performance Boolean operations. Their primary role is to reduce the number of unnecessary intersection tests and spatial queries by hierarchically organizing the geometric domain. By exploiting spatial coherence and locality, such structures enable early pruning of non-overlapping regions and restrict computation to a minimal subset of relevant primitives.

Among the most widely used structures are Binary Space Partitioning (BSP) trees, KD-trees, and octrees. BSP trees, for instance, have been extensively employed in both exact and approximate Boolean frameworks due to their ability to recursively split space along arbitrary planes. Notably, Zhang et al. [7] propose a blending-weight BSP tree construction that optimizes partitioning using shape-aware heuristics, reducing both depth and imbalance.

KD-trees typically operate under axis-aligned constraints and may struggle with oblique or curved geometries [70], especially when tight bounding boxes do not align with model features. This limits their pruning effectiveness in tightly interlocked assemblies or non-axis-aligned shapes. Furthermore, static KD-trees may not adapt well to dynamic

scenes or Boolean operations involving iterative modifications, such as constructive modeling. Octrees and adaptive spatial grids provide another dimension of flexibility. By recursively subdividing space into uniform cubes, octrees facilitate localized intersection computation and are well-suited for volumetric representations. Zhou et al. [11] integrate parallel octree traversal with triangle indexing to enhance performance on high-resolution meshes.

Despite their demonstrated benefits, spatial data structures exhibit varying degrees of effectiveness depending on the input model characteristics and the type of Boolean operation. BSP trees, for instance, offer high precision and flexibility in partitioning, but their construction can be computationally intensive and sensitive to degenerate geometry or skewed distributions of input primitives. In the worst case, poorly balanced BSP trees may incur deep recursion and redundant subdivision, limiting their efficiency in large-scale or highly clustered models. To address this, recent methods [7] incorporate heuristics for plane selection (e.g., minimizing weighted surface area or balancing fragment counts), though such schemes often trade optimality for simplicity in real-time settings.

5.2.3. Parallelism and GPU acceleration

The rapid growth of 3D model complexity has made parallelism one of the most effective strategies for scaling Boolean operations. Modern hardware platforms, ranging from multi-core CPUs to massively parallel GPUs, provide opportunities to accelerate the most expensive stages of the Boolean pipeline, notably intersection computation and classification. Recent research explores both coarse-grained and fine-grained parallelization, as well as heterogeneous CPU/GPU cooperation.

A representative example is QuickCSG [90], which evaluates multi-object CSG expressions by decomposing space into subdomains through a KD-tree and distributing tasks among threads with work-stealing schedulers. This design allows for efficient concurrency at the operation level, demonstrating that Boolean operations over multiple solids can be computed interactively without prohibitive synchronization costs. However, QuickCSG assumes general-position inputs and does not guarantee robustness under degenerate configurations.

Intersection detection is highly amenable to parallelization since triangle-triangle tests can be executed independently. de Magalhães et al. [160] proposed 3D-EPUG-Overlay, a parallel algorithm for computing polygonal mesh overlays with exact arithmetic. By scheduling intersection tests and local updates as independent tasks, their approach achieves substantial speedups while preserving numerical guarantees. Similarly, frameworks such as EMBER [76] leverage recursive spatial subdivision and work-stealing parallelism to accelerate robust local arrangements, demonstrating interactive performance even on models with millions of facets.

While GPUs excel at throughput, exact Boolean computation is challenged by the need for robust predicates and complex topological updates. A practical approach is to use GPUs for accelerating geometric predicates. de Matos Menezes et al. [161] proposed an interval-arithmetic-based method for evaluating exact predicates on GPUs: uncertain cases are filtered back to the CPU for exact rational arithmetic, balancing speed and correctness. Such hybrid CPU-GPU frameworks exploit the strengths of both platforms.

5.2.4. Limitations and discussion

Despite substantial progress in improving the performance of Boolean operations, current efficiency-oriented methods still face persistent bottlenecks. Intersection computation remains the most time-consuming stages when processing large or highly detailed models. Even with spatial partitioning and adaptive refinement, these steps can dominate the total runtime, particularly for models with complex self-intersections or numerous small features. Such bottlenecks limit responsiveness in interactive design systems and hinder the use of Boolean operations in time-critical applications like real-time simulation or large-scale design iteration.

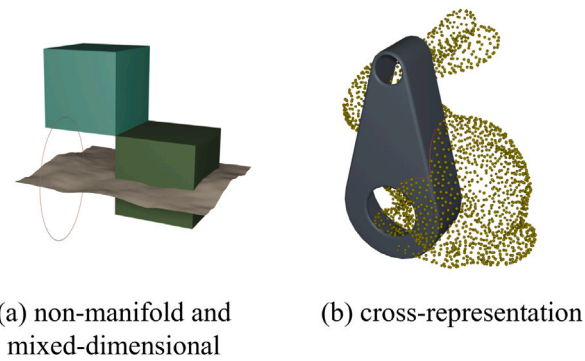


Fig. 16. Examples of input extensibility challenges. (a) Non-manifold and mixed-dimensional inputs, where solids interact with lower-dimensional geometric entities. (b) Cross-representations input, exemplified by combining B-Rep model with point-cloud data.

Parallel and GPU-accelerated methods, while promising, introduce their own limitations. Achieving efficient load balancing is difficult when the workload is highly irregular, as in models with non-uniform geometric complexity. Communication overhead between processing units and synchronization costs can offset the expected speedups, especially in multi-node distributed settings. Moreover, the algorithms that have data dependencies in topological reconstruction or geometric refinement are inherently less amenable to parallelization.

5.3. Extensibility and generalization

Early Boolean algorithms typically imposed strict assumptions about the input data, often requiring manifold polyhedra [28,29,51]. However, contemporary applications [9,27,38,45] have significantly broadened the scope of Boolean operations. These advancements focus on two primary dimensions: input and output extensibility, which entails the ability to handle a variety of model types, including diverse representations, different dimensions, and both manifold and non-manifold models; and semantic extensibility, which broadens Boolean operations beyond traditional set operations to incorporate more complex, attribute-based functionalities.

5.3.1. Input and output extensibility

Input and output extensibility denotes the ability of a Boolean operation algorithm to accommodate heterogeneous or non-standard geometric inputs while producing output representations that remain valid and usable in downstream modeling, analysis, simulation, or fabrication workflows. This requirement becomes particularly important when inputs depart from conventional closed, manifold boundary representations, for example by involving non-manifold topology, mixed-dimensional entities, or multiple geometric representations. Fig. 16 illustrates two representative input-side extensibility challenges: non-manifold and mixed-dimensional configurations, and cross-representation inputs.

Non-manifold and complex geometry handling. Modern geometric modeling increasingly involves inputs that are non-manifold, feature high genus, or contain boundary discontinuities. Early foundational works generalized classical Boolean algebra to explicitly handle open and non-manifold polyhedra [32,34]. Subsequently, Masuda's combinational map data structure systematically incorporated Euler operations supporting non-manifold conditions into industrial-strength Boolean kernels, facilitating practical modeling scenarios involving complex topological entities [39].

Mixed-dimensional geometric models. Emerging industrial design and engineering applications often involve non-manifold and mixed-dimensional configurations, such as solids combined with sheets, wires, or isolated lower-dimensional features. Early work by Gursoz et al. [38] addressed Boolean set operations on non-manifold boundary-representation objects, providing an important foundation for handling heterogeneous geometric entities within a unified topological setting. Similarly, Granados et al. [16] showed that Nef-polyhedron-based representations can support non-manifold, open/closed, and mixed-dimensional complexes, enabling robust Boolean operations on inputs that may contain polygonal facets together with isolated edges and vertices. Beyond the three-dimensional setting, Paoluzzi et al. [162] proposed a progressive, dimension-independent framework for Boolean operations on polyhedral objects of arbitrary dimension.

Hybrid representation integration. To leverage complementary strengths of various geometric representations, hybrid frameworks increasingly convert input surfaces into volumetric or implicit forms for Boolean computations, subsequently reconstructing accurate output representations. Biermann et al. [44] proposed an influential hybrid pipeline that utilizes rasterization to voxel-based implicit representations for robust Boolean computations, with subsequent error-controlled surface reconstruction ensuring geometric fidelity. Fougere et al. [91] further integrated implicit fields and parametric surfaces using R-functions, achieving smooth (C^1 -continuous) interfaces across different representations. Adams and Dutré [81] notably extended hybrid approaches to point-based surface geometry, demonstrating robustness and interactivity through GPU-accelerated hybrid processing pipelines.

5.3.2. Semantic extensibility of Boolean operations

Attribute-enriched Boolean operations. Classical Boolean operators traditionally focus on geometry and topology. In heterogeneous object modeling, however, geometric entities may also carry material or compositional attributes, requiring Boolean operators that preserve or recompute these attributes over the resulting domain. Sun and Hu [45] introduced reasoning Boolean operations for heterogeneous objects, including material-dominant union, subtraction, and intersection, together with a complex-union operator for assembling heterogeneous results. Related work in heterogeneous solid modeling further supports Boolean updates with explicit handling of material-variation attributes in boundary representations [163].

Extended Boolean operators. Beyond conventional regularized Boolean operations (union, intersection, and difference), later work extended Boolean modeling in several distinct directions. In implicit modeling, Wyvill et al. [83] introduced the BlobTree, in which blending, warping, and Boolean operations are all represented as nodes in a unified tree structure, enabling smooth combinations of implicit primitives and supporting applications such as shape blending and organic modeling. Their framework is related to earlier R-function-based formulations, but is developed in the context of skeleton-based implicit surfaces. Pereira et al. [59] extended Boolean operations to multi-region and non-manifold solids for mesh generation, allowing the treatment of internal interfaces and degenerate components such as shells and wires. More recently, Paoluzzi and Scorzelli [50] reformulated solid modeling in an algebraic-topological setting based on Boolean algebras, spatial arrangements, and chain complexes, providing support for n-ary Boolean expressions and a more general theoretical treatment of solid operations.

Differentiable and neural-based Boolean operations. Emerging differentiable modeling paradigms require Boolean operations amenable to gradient-based optimization. Field-based implicit formulations using smooth R-functions inherently satisfy differentiability constraints, enabling optimization-driven modeling workflows [84]. Recent advances further embed differentiable Boolean operators within neural implicit representations, facilitating gradient-based shape optimization and inverse design scenarios. For instance, Liu et al. [9] propose a unified

differentiable Boolean operator grounded in fuzzy logic that supports continuous interpolation among union, intersection, and difference. This operator is differentiable not only with respect to the operands but also to the operation type itself, enabling direct optimization over both primitive parameters and Boolean structure within CSG trees. Their framework enables efficient inverse CSG fitting via gradient descent and supports smooth transitions between hard and soft shape compositions, showing improved reconstruction accuracy and compactness over traditional formulations. Such neural CSG approaches demonstrate promising scalability and real-time inference capabilities, significantly expanding potential applications in computational design and simulation-driven optimization.

5.3.3. Limitations and discussion

To broaden applicability and support increasingly diverse modeling scenarios, Boolean operations need to evolve toward more general and extensible frameworks. Despite encouraging progress, several important limitations still constrain such development.

First, building unified data structures that can simultaneously manage multiple geometric representations, such as meshes, volumetric grids, parametric surfaces, and implicit fields, remains an open problem. Current solutions often rely on repeated conversions or hybrid intermediates, which increase computational cost and reduce numerical robustness [27,94]. The lack of a shared abstraction makes it difficult to achieve smooth interoperability across heterogeneous modeling pipelines.

Second, existing frameworks still provide limited support for multi-attribute propagation. Practical models usually carry diverse information beyond geometry, including material parameters, colors, or physical properties. Efficiently handling and preserving these attributes throughout intersection, classification, and reconstruction processes would greatly improve usability in engineering and visualization applications [45]. However, consistent attribute blending and inheritance are not yet well established, often requiring additional manual adjustments.

Third, emerging differentiable Boolean operators based on neural implicit representations and smooth analytical formulations offer new opportunities for optimization-driven design [9]. These methods enable gradient-based learning and inverse modeling but currently face trade-offs between differentiability and exactness. Preserving sharp boundaries and topological integrity while maintaining differentiability remains a technical challenge.

Overall, further progress will likely rely on integrative abstraction layers that can represent different geometric forms under a common computational model, while also maintaining consistency of semantic and physical attributes. Instead of pursuing a single universal solution, future research may move toward modular and interoperable systems, where discrete, parametric, and implicit representations can interact more flexibly within shared frameworks. Such developments could gradually improve the extensibility of Boolean operations, enabling them to adapt more naturally to diverse modeling contexts without compromising robustness or precision.

6. Conclusion

Boolean operations serve as a foundational component of 3D geometric modeling, enabling the construction of complex shapes via set-theoretic manipulations such as union, intersection, and difference. Despite decades of research and development, achieving robust, efficient, and extensible Boolean evaluation remains a persistent challenge, particularly in the face of numerical instability, topological inconsistencies, and heterogeneous input representations.

This survey provides a comprehensive and structured review of 3D Boolean operations. To the best of our knowledge, this is the first dedicated, standalone survey on this topic. We adopt a common four-stage computational framework, comprising intersection computation,

geometric splitting, fragment classification, and topological reconstruction, which captures the common algorithmic patterns underlying most Boolean implementations. This abstraction not only enables systematic comparison across methods, but also clarifies how representation-specific strategies address fundamental design trade-offs.

In addition to classical explicit representations, the survey explores Boolean operations for other representations such as CSG, BSP, volumetric representation, implicit representation, and hybrid representation. These representations offer promising routes toward improved robustness and generalization, especially in contexts involving noisy, non-manifold, or mixed-representation input data.

Furthermore, we analyze core challenges in three dimensions: robustness and numerical stability, algorithmic efficiency and scalability, and extensibility to broader semantic. For each axis, we review representative strategies and identify open problems that merit further investigation.

By consolidating theoretical foundations, algorithmic methodologies, and practical advances, this survey aims to serve as a reference for both researchers and practitioners seeking to deepen their understanding or contribute to the development of reliable and scalable Boolean modeling techniques.

CRedit authorship contribution statement

Sili Liang: Writing – review & editing, Writing – original draft, Investigation. **Anchang Bao:** Writing – review & editing. **Enya Shen:** Writing – review & editing. **Jianmin Wang:** Writing – review & editing.

Author Use of AI Tools

During the preparation of this manuscript, generative AI tools (e.g., ChatGPT by OpenAI) were used to assist with improving language and grammar. All content was reviewed and edited by the authors to ensure accuracy and originality. The authors take full responsibility for the content of this manuscript.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work has been funded by the National Key R&D Program of China (No. 2023YFB3309000) and the Shenzhen Innovation and Entrepreneurship Program (Nos. KJZD20230923114114028 and 20232910020).

Data availability

No data was used for the research described in the article.

References

- [1] Sarcar M, Rao KM, Narayan KL. Computer aided design and manufacturing. PHI Learning Pvt. Ltd.; 2008.
- [2] Foley JD. In: Computer graphics: principles and practice, vol. 12110, Addison-Wesley Professional; 1996.
- [3] Tost D, Puig A, Pérez-Vidal L. Boolean operations for 3D simulation of CNC machining of drilling tools. *Computer-Aided Des* 2004;36:315–23.
- [4] Charton J, Laurentjoye M, Kim Y. 3D Boolean operations in virtual surgical planning. *Int J Comput Assist Radiol Surg* 2017;12:1697–709.
- [5] Yang Y, Jia X, Wang B, Yang J, Xin S, Yan DM. Boolean operation for cad models using a hybrid representation. *ACM Trans Graph* 2025;44.
- [6] Xiao Y, Chen M, Chen S, Lu S. Faster than fast: Efficient approximate boolean operations on dense triangular mesh models. *Computer-Aided Des Appl* 2025;1007–26. <http://dx.doi.org/10.14733/cadaps.2025.1007-1026>.
- [7] Zhang H, Song T, Zhang G, Dai K, Wang L, Liu Y, Yang YJ, Zeng W. Blending weight BSP tree for mesh Boolean operations. *Computer-Aided Des* 2025;182:103849.
- [8] Guo JP, Fu XM. Exact and efficient intersection resolution for mesh arrangements. *ACM Trans Graph* 2024;43:1–14.
- [9] Liu HTD, Agrawala M, Yuksel C, Omernick T, Misra V, Corazza S, McGuire M, Zordan V. A unified differentiable boolean operator with fuzzy logic. In: Special interest group on computer graphics and interactive techniques conference papers '24. 2024, p. 1–9, arXiv:2407.10954.
- [10] Yanxi L, Dongmei Z, Xiaohui Z, Xu X, Ketai H. Exact boolean operation on triangulated meshes using accelerated BSPs. In: 2023 IEEE 18th conference on industrial electronics and applications. Ningbo, China: IEEE; 2023, p. 1662–7.
- [11] Zhou M, Qin J, Mei G, Tipper JC. Simple and robust Boolean operations for triangulated surfaces. *Mathematics* 2023;11:2713.
- [12] Paoluzzi A, Shapiro V, DiCarlo A, Scorzelli G, Onofri E. Finite algebras for solid modeling using Julia's sparse arrays. *Computer-Aided Des* 2023;155:103436.
- [13] Siemens Digital Industries Software. Parasolid 3D geometric modeling kernel. 2025, URL <https://plm.sw.siemens.com/en-US/plm-components/parasolid/>, used by various CAD/CAM/CAE applications.
- [14] Corp. S. ACIS Geometric Modeler. Broomfield, CO: Spatial Corp.; 2025, URL <https://www.spatial.com/products/3d-acis-modeling>, version 2025.
- [15] Open Cascade SAS. Open CASCADE technology (OCCT). 2025, URL <https://dev.opencascade.org>, version 7.8.0.
- [16] Granados M, Hachenberger P, Hert S, Kettner L, Mehlhorn K, Seel M. Boolean operations on 3D selective Nef complexes: Data structure, algorithms, and implementation. In: Goos G, Hartmanis J, Van Leeuwen J, Di Battista G, Zwick U, editors. In: Algorithms - ESA 2003, vol. 2832, Berlin, Heidelberg: Springer Berlin Heidelberg; 2003, p. 654–66.
- [17] Hachenberger P, Kettner L. Boolean operations on 3D selective Nef complexes: Optimized implementation and experiments. In: Proceedings of the 2005 ACM symposium on solid and physical modeling. New York, NY, USA: Association for Computing Machinery; 2005, p. 163–74.
- [18] Hoffmann CM. Geometric and solid modeling: An introduction. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc.; 1989.
- [19] Stroud I. Boundary representation modelling techniques. Springer; 2006.
- [20] Voelcker H, Requicha A. Constructive solid geometry. 1977.
- [21] Braid I. Designing with volumes. University of Cambridge Computer Laboratory ComputerAided Design Group; 1973, URL <https://books.google.co.jp/books?id=E5ywAAAAIAAJ>.
- [22] Botsch M, Kobbelt L, Pauly M, Alliez P, Lévy B. Polygon mesh processing. CRC Press; 2010.
- [23] Giraud C. Presque half-planes: Towards a general representation scheme. *Computer-Aided Des* 1984;16:17–24.
- [24] Pasko A, Adzhiev V, Sourin A, Savchenko V. Function representation in geometric modeling: Concepts, implementation and applications. *Vis Comput* 1995;11:429–46.
- [25] Bloomenthal J, Bajaj C. Introduction to implicit surfaces. Morgan Kaufmann; 1997.
- [26] Meagher D. Geometric modeling using octree encoding. *Comput Graph Image Process* 1982;19:129–47.
- [27] Sheng B, Liu B, Li P, Fu H, Ma L, Wu E. Accelerated robust Boolean operations based on hybrid representations. *Comput Aided Geom Design* 2018;62:133–53.
- [28] Requicha A, Voelcker H. Boolean operations in solid modeling: Boundary evaluation and merging algorithms. *Proc IEEE* 1985;73:30–44.
- [29] Mäntylä M. Boolean operations of 2-manifolds through vertex neighborhood classification. *ACM Trans Graph* 1986;5:1–29.
- [30] Putnam L, Subrahmanyam P. Boolean operations on n-Dimensional objects. *IEEE Comput Graph Appl* 1986;6:43–51.
- [31] Farouki RT. Truncated-surface algorithms for the evaluation and interrogation of solid boundary representations. *IBM J Res Dev* 1987;31:314–34.
- [32] Bieri H, Nef W. Elementary set operations with d-dimensional polyhedra. In: Computational geometry and its applications: CG'88, international workshop on computational geometry wüzburg, FRG, March 24–25, 1988 proceedings. Springer; 1988, p. 97–112.
- [33] Casale MS, Bobrow JE. A set operation algorithm for sculptured solids modeled with trimmed patches. *Comput Aided Geom Design* 1989;6:235–47.
- [34] Paoluzzi A, Ramella M, Santarelli A. Boolean algebra over linear polyhedra. *Computer-Aided Des* 1989;21:474–84.
- [35] Hoffmann C, Hopcroft J, Karasick M. Robust set operations on polyhedral solids. *IEEE Comput Graph Appl* 1989;9:50–9.
- [36] Toriya H, Takamura T, Satoh T, Chiyokura H. Boolean operations of solids with free-form surfaces through polyhedral approximation. In: New advances in computer graphics: proceedings of CG international'89. Springer; 1989, p. 405–20.
- [37] Satoh T, Chiyokura H. Boolean operations on sets using surface data. In: Proceedings of the first ACM symposium on solid modeling foundations and CAD/CAM applications - SMA '91. Austin, Texas, United States: ACM Press; 1991, p. 119–26.
- [38] Guroz E, Choi Y, Prinz FB. Boolean set operations on non-manifold boundary representation objects. *Computer-Aided Des* 1991;23:33–9.

- [39] Masuda H. Topological operators and Boolean operations for complex-based nonmanifold geometric models. *Computer-Aided Des* 1993;25:119–29.
- [40] Earnst PH. Boolean set operations with solid models. *Hewlett Packard J* 1995;46: 74–74.
- [41] Gardan Y, Perrin E. An algorithm reducing 3D Boolean operations to a 2D problem: Concepts and results. *Computer-Aided Des* 1996;28:277–87.
- [42] Hui KC, Kan YM. Distributed boundary representation and Boolean operations on a massively parallel computer. *Computer-Aided Des* 1996;28:631–49.
- [43] Krishnan S, Manocha D, Gopi M, Culver T, Keyser J. Boole: A boundary evaluation system for boolean combinations of sculptured solids. *Internat J Comput Geom Appl* 2001;11:105–44.
- [44] Biermann H, Kristjansson D, Zorin D. Approximate Boolean operations on free-form solids. In: *Proceedings of the 28th annual conference on computer graphics and interactive techniques*. New York, NY, USA: Association for Computing Machinery; 2001, p. 185–94.
- [45] Sun W, Hu X. Reasoning Boolean operation based modeling for heterogeneous objects. *Computer-Aided Des* 2002;34:481–8.
- [46] Keyser J, Culver T, Foskey M, Krishnan S, Manocha D. ESOLID—a system for exact boundary evaluation. *Computer-Aided Des* 2004;36:175–93.
- [47] García Á, Ruiz de Miras J, Feito FR. Evaluation of boolean operations between free-form solids using extended simplicial chains and pn triangles. *Vis Comput* 2011;27:531–41.
- [48] Xu S, Keyser J. Fast and robust Booleans on polyhedra. *Computer-Aided Des* 2013;45:529–34.
- [49] Urick B, Marussig B, Cohen E, Crawford RH, Hughes TJ, Riesenfeld RF. Watertight Boolean operations: A framework for creating CAD-compatible gap-free editable solid models. *Computer-Aided Des* 2019;115:147–60.
- [50] Paoluzzi A, Scorzelli G. Computational topology, Boolean algebras, and solid modeling. *Computer-Aided Des* 2025;181:103839.
- [51] Yamaguchi F, Tokieda T. A unified algorithm for boolean shape operations. *IEEE Comput Graph Appl* 1984;4:24–37.
- [52] Ogayar CJ, Feito FR, Segura RJ, Rivero M. Gpu-based evaluation of boolean operations on triangulated solids. 2006.
- [53] Smith J, Dodgson N. A topologically robust algorithm for Boolean operations on polyhedral shapes using approximate arithmetic. *Computer-Aided Des* 2007;39:149–63.
- [54] Yongbin J, Liguang W, Lin B, Jianhong C. Boolean operations on polygonal meshes using OBB trees. In: *2009 international conference on environmental science and information application technology*. Wuhan, China: IEEE; 2009, p. 619–22.
- [55] Schifko M, Jüttler B, Kornberger B. Industrial application of exact Boolean operations for meshes. In: *Proceedings of the 26th spring conference on computer graphics*. Budmerice Slovakia: ACM; 2010, p. 165–72.
- [56] Campen M, Kobbelt L. Exact and robust (self-)intersections for polygonal meshes. *Comput Graph Forum* 2010;29:397–406.
- [57] Tayebi A, Pérez JG, Diego IG, Cátedra F. Boolean operations implementation over 3D parametric surfaces to be included in the geometrical module of an electromagnetic solver.
- [58] Wang CCL. Approximate Boolean operations on large polyhedral solids with partial mesh reconstruction. *IEEE Trans Vis Comput Graphics* 2011b;17:836–49.
- [59] Pereira AMB, De Arruda MC, Miranda ACDO, Lira WWM, Martha LF. Boolean operations on multi-region solids for mesh generation. *Eng Comput* 2012;28:225–39.
- [60] Feito F, Ogayar C, Segura R, Rivero M. Fast and accurate evaluation of regularized Boolean operations on triangulated solids. *Computer-Aided Des* 2013;45:705–16.
- [61] Barki H, Guennebaud G, Foufou S. Exact, robust, and efficient regularized Booleans on general 3D meshes. *Comput Math Appl* 2015;70:1235–54.
- [62] Jacobson A. Boolean operations using generalized winding numbers. 2016, arXiv:1601.07953.
- [63] Xiao Z, Chen J, Zheng Y, Zheng J, Wang D. Booleans of triangulated solids by a boundary conforming tetrahedral mesh generation approach. *Comput Graph* 2016.
- [64] Schmidt R, Brochu T. Adaptive mesh booleans. 2016, arXiv:1605.01760.
- [65] Zhou Q, Grinspun E, Zorin D, Jacobson A. Mesh arrangements for solid geometry. *ACM Trans Graph* 2016;35:1–15.
- [66] Landier S. Boolean operations on arbitrary polygonal and polyhedral meshes. *Computer-Aided Des* 2017;85:138–53.
- [67] Qin Y, Luo Z, Wen L, Feng C, Zhang X, Lan M, Liu B. Research and application of Boolean operation for triangular mesh model of underground space engineering—Boolean operation for triangular mesh model. *Energy Sci Eng* 2019;7:1154–65.
- [68] Wang H, Kan S, Zhang X, Lu X, Zhou L. Robust Boolean operations algorithm on regularized triangular mesh and implementation. *Multimedia Tools Appl* 2020;79:5301–20.
- [69] Cherchi G, Livesu M, Scateni R, Attene M. Fast and robust mesh arrangements using floating-point arithmetic. *ACM Trans Graph* 2020;39:1–16.
- [70] Cherchi G, Pellacini F, Attene M, Livesu M. Interactive and robust mesh booleans. 2022, arXiv:2205.14151.
- [71] Zheng W, Wu H, Xu G, Ling R, Gu R. Feature-preserving quadrilateral mesh Boolean operation with cross-field guided layout blending. *Comput Aided Geom Design* 2024;111:102324.
- [72] Thibault WC, Naylor BF. Set operations on polyhedra using binary space partitioning trees. In: *Proceedings of the 14th annual conference on computer graphics and interactive techniques*. New York, NY, USA: Association for Computing Machinery; 1987, p. 153–62.
- [73] Naylor B, Amanatides J, Thibault W. Merging BSP trees yields polyhedral set operations. *SIGGRAPH Comput Graph* 1990;24:115–24.
- [74] Bernstein G, Fussell D. Fast, exact, linear booleans. *Comput Graph Forum* 2009;28:1269–78.
- [75] Nehring-Wirxel J, Trettner P, Kobbelt L. Fast exact booleans for iterated CSG using octree-embedded bsps. *Computer-Aided Des* 2021;135:103015.
- [76] Trettner P, Nehring-Wirxel J, Kobbelt L. EMBER: Exact mesh booleans via efficient & robust local arrangements. *ACM Trans Graph* 2022;41:1–15.
- [77] Ayala D, Brunet P, Juan R, Navazo I. Object representation by means of nonminimal division quadrees and octrees. *ACM Trans Graph* 1985;4:41–59.
- [78] Carlbom I. An algorithm for geometric set operations using cellular subdivision techniques. *IEEE Comput Graph Appl* 1987;7:44–55.
- [79] Ayala D. Boolean operations between solids and surfaces by octrees: Models and algorithm. *Computer-Aided Des* 1988;20:452–65.
- [80] Brunet P, Navazo I. Solid representation and operation using extended octrees. *ACM Trans Graph* 1990;9:170–97.
- [81] Adams B, Dutré P. Interactive boolean operations on surfel-bounded solids. *ACM Trans Graph* 2003;22:651–6.
- [82] Wang CC. Approximate boolean operations on large polyhedral solids with partial mesh reconstruction. *IEEE Trans Vis Comput Graphics* 2011a;17:836–49.
- [83] Wyvill B, Guy A, Galin E. Extending the CSG tree. Warping, blending and Boolean operations in an implicit surface modeling system. *Comput Graph Forum* 1999;18:149–58.
- [84] Qin X, Yang X, Zheng H. Implicit surface Boolean operations based cut-and-paste algorithm for mesh models. In: Pan Z, Cheok A, Haller M, Lau RWH, Saito H, Liang R, editors. *In: Advances in artificial reality and tele-existence*, vol. 4282, Berlin, Heidelberg: Springer Berlin Heidelberg; 2006, p. 839–48.
- [85] Gourmel O, Barthe L, Cani MP, Wyvill B, Bernhardt A, Paulin M, Grasberger H. A gradient-based implicit blend. *ACM Trans Graph* 2013;32:1–12.
- [86] Chen X, Hou F, Wang W, Qin H, He Y. Mind: Material interface generation from udfs for non-manifold surface reconstruction. 2025, arXiv preprint arXiv: 2506.02938.
- [87] Bronsvort WF. Techniques for reducing Boolean evaluation time in CSG scan-line algorithms. *Computer-Aided Des* 1986;18:533–6.
- [88] Hable J, Rossignac J. Blister: GPU-based rendering of Boolean combinations of free-form triangulated shapes. *ACM Trans Graph* 2005;24:1024–31.
- [89] Rossignac J, Fudos I, Vasilakis A. Direct rendering of Boolean combinations of self-trimmed surfaces. *Computer-Aided Des* 2013;45:288–300.
- [90] Douze M, Franco JS, Raffin B. QuickCSG: Fast arbitrary Boolean combinations of N solids, arXiv:1706.01558.
- [91] Fougerolle Y, Gribok A, Foufou S, Truchetet F, Abidi M. Boolean operations with implicit and parametric representation of primitives using R-functions. *IEEE Trans Vis Comput Graphics* 2005;11:529–39.
- [92] Yang P, Qian X. Direct boolean intersection between acquired and designed geometry. *Computer-Aided Des* 2009;41:81–94.
- [93] Pavić D, Campen M, Kobbelt L. Hybrid booleans. In: *Computer graphics forum*. Wiley Online Library; 2010, p. 75–87.
- [94] Upreti K, Subbarayan G. Signed algebraic level sets on NURBS surfaces and implicit Boolean compositions for isogeometric CAD–CAE integration. *Computer-Aided Des* 2017;82:112–26.
- [95] Okino N. Tips-1, technical information processing system for computer aided design, drawing and manufacturing. In: *Proc. PROLAMAT'73*. 1973.
- [96] Braid I, of Cambridge U. Notes on a geometric modeller. CAD Group Document, University of Cambridge; 1979, URL <https://books.google.co.jp/books?id=elbbPwAACAAJ>.
- [97] Spatial Corp. Cgm core modeler: Convergence geometric modeler. 2026, <https://www.spatial.com/solutions/3d-modeling/cgm-modeler>, Geometric modeling kernel used in Dassault Systèmes products.
- [98] PTC Inc. Ptc granite geometric kernel. 2026, URL <https://www.ptc.com/.geometric-modelling-kernel-developed-by-ptc>; (Accessed 11 February 2026).
- [99] Xie Y, Takikawa T, Saito S, Litany O, Yan S, Khan N, Tombari F, Tompkin J, Sitzmann V, Sridhar S. Neural fields in visual computing and beyond. In: *Computer graphics forum*. Wiley Online Library; 2022, p. 641–76.
- [100] Lorens WE, Cline HE. Marching cubes: A high resolution 3d surface construction algorithm. In: *Seminal graphics: pioneering efforts that shaped the field*. 1998, p. 347–53.
- [101] Newman TS, Yi H. A survey of the marching cubes algorithm. *Comput Graph* 2006.
- [102] Requicha AG. Representations for rigid solids: Theory, methods, and systems. *ACM Comput Surv* 1980;12:437–64.
- [103] Astarloğlu E. Comparing boolean operation methods on 3D solids [Master's thesis], Aalto University; 2023, URL <https://aaltodoc.aalto.fi/items/25f3c135-b4db-4af5-93d4-388661594e5b>.

- [104] Tilove. Set Membership Classification: A Unified Approach to Geometric Intersection Problems. *IEEE Trans Comput* 1980;C-29:874–83.
- [105] Tilove R. A study of geometric set-membership classification. 1977.
- [106] Hubbard PM. Interactive collision detection. In: *Proceedings of 1993 IEEE research properties in virtual reality symposium*. IEEE; 1993, p. 24–31.
- [107] Kockara S, Halic T, Iqbal K, Bayrak C, Rowe R. Collision detection: A survey. In: *2007 IEEE international conference on systems, man and cybernetics*. IEEE; 2007, p. 4046–51.
- [108] Sutherland IE, Hodgman GW. Reentrant polygon clipping. *Commun ACM* 1974;17:32–42.
- [109] Weiler K, Atherton P. Hidden surface removal using polygon area sorting. *ACM SIGGRAPH Comput Graph* 1977;11:214–22.
- [110] Möller T. A Fast Triangle-Triangle Intersection Test. *J Graph Tools* 1997;2:25–30.
- [111] Guigue P, Devillers O. Fast and robust triangle-triangle overlap test using orientation predicates. *J Graph Tools* 8:25–32, URL http://www.philippe-guigue.de/data/triangle_triangle_intersection.html.
- [112] Piegli L, Tiller W. *The NURBS book*. Springer Science & Business Media; 2012.
- [113] Patrikalakis NM, Maekawa T. In: *Shape interrogation for computer aided design and manufacturing*, vol. 15, Springer; 2002.
- [114] Patrikalakis NM. Surface-to-surface intersections. *IEEE Comput Graph Appl* 2002;13:89–95.
- [115] Cheng JS, Zhang B, Xiao Y, Li M. Topology driven approximation to rational surface-surface intersection via interval algebraic topology analysis. *ACM Trans Graph* 2023;42:1–16.
- [116] Patrikalakis NM, Maekawa T. Intersection problems. In: *Shape interrogation for computer aided design and manufacturing*. Springer; 2009, p. 109–60.
- [117] Zhang B, Cheng JS, Liu YS, Zhang Z. Topology guaranteed and error controlled curve tracing for parametric surface-surface intersection. *Comput Aided Geom Design* 2025;118:102432.
- [118] Shao W, Chen F, Liu X. Robust algebraic curve intersections with tolerance control. *Computer-Aided Des* 2022;147:103236.
- [119] Chen H, Chen XD. An efficient and robust tracing method based on matrix representation for surface-surface intersection. In: *International conference on computational visual media*. Springer; 2025, p. 49–68.
- [120] Manocha D, Demmel J. Algorithms for intersecting parametric and algebraic curves i: simple intersections. *ACM Trans Graph* 1994;13:73–100.
- [121] Wintz J. *Algebraic methods for geometric modeling* (Ph.D. thesis), Université Nice Sophia Antipolis; 2008.
- [122] Barnhill RE, Farin G, Jordan M, Piper BR. Surface/surface intersection. *Comput Aided Geom Design* 1987;4:3–16.
- [123] Livesu M, Cherchi G, Scateni R, Attene M. Deterministic Linear Time Constrained Triangulation Using Simplified Earcut. *IEEE Trans Vis Comput Graphics* 2022;28:5172–7.
- [124] Feito FR, Torres JC. Inclusion test for general polyhedra. *Comput Graph* 1997.
- [125] Ogayar CJ, Segura RJ, Feito FR. Point in solid strategies. *Comput Graph* 2005.
- [126] Barill G, Dickson NG, Schmidt R, Levin DIW, Jacobson A. Fast winding numbers for soups and clouds. *ACM Trans Graph* 2018;37.
- [127] Barnes J, Hut P. A hierarchical $O(n \log n)$ force-calculation algorithm. *Nature* 1986;324:446–9.
- [128] Madan A, Sharp N, Williams F, Museth K, Levin DIW. Stochastic Barnes-Hut approximation for fast summation on the GPU. 2025, <http://dx.doi.org/10.1145/3721238.3730725>, arXiv:2506.02219.
- [129] Attene M, Campen M, Kobbelt L. Polygon mesh repairing: An application perspective. *ACM Comput Surv* 2013;45:1–33.
- [130] Zhou L, Wang D, Sheng Y, Wang Y, Li P. Three dimensional Boolean operation based on L-Rep model. In: *2010 18th international conference on geoinformatics*. Beijing, China: IEEE; 2010, p. 1–6.
- [131] Su CJ, Lin F, Ye L. A new collision detection method for csg-represented objects in virtual manufacturing. *Comput Ind* 1999;40:1–13.
- [132] Yuan H, Bousseau A, Pan H, Zhang C, Mitra NJ, Li C. DiffCSG: Differentiable CSG via rasterization. 2024, arXiv:2409.01421.
- [133] Museth K, Lait J, Johanson J, Budsberg J, Henderson R, Alden D, Cucka P, Hill D, Pearce A. Openvdb: an open-source data structure and toolkit for high-resolution volumes. In: *Acm siggraph 2013 courses*. 2013, 1–1.
- [134] Park JJ, Florence P, Straub J, Newcombe R, Lovegrove S. DeepSDF: Learning continuous signed distance functions for shape representation. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2019, p. 165–74.
- [135] Mescheder L, Oechsle M, Niemeyer M, Nowozin S, Geiger A. Occupancy networks: Learning 3d reconstruction in function space. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2019, p. 4460–70.
- [136] Ricci A. A constructive geometry for computer graphics. *Comput J* 1973;16:157–60.
- [137] nTopology I. ntopology. 2026, URL <https://ntop.com/>.
- [138] Inc. A. Adobe substance 3d. 2026, URL <https://substance3d.adobe.com/>.
- [139] Womp I. Womp 3d creator. 2026, URL <https://www.womp.com/>.
- [140] Community M. Magicacsg. 2026, URL <https://ephtracy.github.io/index.html?page=magicacsg>.
- [141] Richard Shewchuk J. Adaptive precision floating-point arithmetic and fast robust geometric predicates. *Discrete Comput Geom* 1997;18:305–63. <http://dx.doi.org/10.1007/PL00009321>.
- [142] Fortune S. Polyhedral modelling with multiprecision integer arithmetic. *Computer-Aided Des* 1997;29:123–33.
- [143] Sugihara K. Robust geometric computation based on topological consistency. In: *Alexandrov VN, Dongarra JJ, Juliano BA, Renner RS, Tan CJK, editors. Computational science — ICCS 2001*. Berlin, Heidelberg: Springer Berlin Heidelberg; 2001, p. 12–26.
- [144] Granlund T. GNU MP 6.0 Multiple precision arithmetic library. Samurai Media Limited; 2015.
- [145] Fousse L, Hanrot G, Lefèvre V, Pélissier P, Zimmermann P. Mpmc: A multiple-precision binary floating-point library with correct rounding. *ACM Trans Math Softw (TOMS)* 2007;33:13–es.
- [146] Pion S, Fabri A. A generic lazy evaluation scheme for exact geometric computations. *Sci Comput Program* 2011;76:307–23.
- [147] Sharma V, Yap CK. Robust geometric computation. In: *Handbook of discrete and computational geometry*. Chapman and Hall/CRC; 2017, p. 1189–223.
- [148] Fabri A, Giezeman GJ, Kettner L, Schirra S, Schönherr S. On the design of cgal a computational geometry algorithms library. *Softw: Pr Exp* 2000;30:1167–202.
- [149] Burnikel C, Funke S, Seel M. Exact geometric computation using cascading. *Internat J Comput Geom Appl* 2001;11:245–66.
- [150] Devillers O, Preparata FP. Further results on arithmetic filters for geometric predicates. *Comput Geom* 1999;13:141–8.
- [151] de Matos Menezes M, Magalhães SVG, Franklin WR, de Oliveira MA, Chichorro REB. Accelerating the exact evaluation of geometric predicates with gpus. In: *28th international meshing roundtable*, Buffalo, NY, USA, vol. 16, 2019.
- [152] Bartels T, Fisikopoulos V, Weiser M. Fast floating-point filters for robust predicates. *BIT Numer Math* 2023;63:31.
- [153] Attene M. Indirect predicates for geometric constructions. *Computer-Aided Des* 2020;126:102856.
- [154] Schirra S. Robustness and precision issues in geometric computation. 1998.
- [155] Hong Y, Chang T. A comprehensive review of tolerancing research. *Int J Prod Res* 2002;40:2425–59.
- [156] Salesin D, Stolfi J, Guibas L. Epsilon geometry: building robust algorithms from imprecise computations. In: *Proceedings of the fifth annual symposium on computational geometry*. 1989, p. 208–17.
- [157] Qi J, Shapiro V. Topological formulation of tolerant solid modeling. *Computer-Aided Des* 2006;38:367–77.
- [158] Goodrich MT, Guibas LJ, Hershberger J, Tanenbaum PJ. Snap rounding line segments efficiently in two and three dimensions. In: *Proceedings of the thirteenth annual symposium on computational geometry*. New York, NY, USA: Association for Computing Machinery; 1997, p. 284–93. <http://dx.doi.org/10.1145/262839.262985>.
- [159] Moore RE, Kearfott RB, Cloud MJ. *Introduction to interval analysis*. SIAM; 2009.
- [160] de Magalhães SVG, Franklin WR, Andrade MVA. An efficient and exact parallel algorithm for intersecting large 3-D triangular meshes using arithmetic filters. *Computer-Aided Des* 2020;120:102801.
- [161] de Matos Menezes M, Viana Gomes de Magalhães S, Aguiar de Oliveira M, Randolph Franklin W, de Oliveira Bauer Chichorro RE. Fast parallel evaluation of exact geometric predicates on gpus. *Computer-Aided Des* 2022;150:103285.
- [162] Paoluzzi A, Pascucci V, Scorzelli G. Progressive dimension-independent boolean operations. In: *Elber G, Patrikalakis N, Brunet P, editors. Solid modeling. The Eurographics Association; 2004*, <http://dx.doi.org/10.2312/sm.20041391>.
- [163] Qian Xiaoping, Dutta Deba. Design of heterogeneous turbine blade. *Computer-aided design* 2003;35(3):319–29.